

National Institute of Applied Sciences and Technology

UNIVERSITY OF CARTHAGE

End of Studies Project

Major : **Software Engineering**

Designing and Implementing a Low-code ultra-flexible turnkey digital solution

Presented by

Mohamed MISKINI

INSAT's Supervisor : **Ms. HAMDI Sana**
Company Mentors : **Mr. JOHANN Alexandre**
Mr. CHAOUI Younes

Presented In : **October 2020**

Academic Year : 2019/2020

Dedications

I dedicate this work to my family.
For their unconditional love and support.

I dedicate this work to my friends.
For their words of encouragement and tenacity.

I dedicate this work to my Supervisors.
For their helpful insights and thorough guidance.

I dedicate this work to my Institute and Professors.
For the endless knowledge I acquired both in and out of the classroom.

And Finally, I dedicate this work to the IT field.
Hoping I contributed in any way to have an impact.

Acknowledgments

I would like to thank all of those who contributed to the completion of this work both in a direct or an indirect manner , ensuring its steady progress especially under the abnormal conditions of the global pandemic.

I express my deepest gratitude to the company in which I had my internship for the rich experience and the challenging opportunity I had been offered. A special thanks to **Mr. Younes Chaoui** and **Mr. Alexandre Johann** for their helpful insights and continuous supervision despite their busy schedules.

I would also like to thank my academic supervisor **Ms. Sana Hamdi** for her valuable and concise guidance and her knowledge which allowed me to greatly elaborate this work.

I am very grateful for the members of the honorable **jury** for taking the time to evaluate my work and provide their priceless feedback.

Finally, I wish to thank the school division and all my professors for the quality of teaching and the robustness of the learning process.

Table of contents

List of Figures vi

List of Tables viii

List of Acronyms ix

Abstract x

Résumé xi

General Introduction 1

I Project Context 2

1 Presentation of the Host Company . . . 3
1.1 Kostango . . . 3
1.2 Areas of Expertise . . . 3
1.3 Notable Customers . . . 5
1.4 Hierarchical Structure . . . 5
2 Presentation of the Project . . . 6
2.1 Project Definition . . . 6
2.2 Motivations . . . 6
2.2.1 Traditional Tools . . . 6
2.2.2 Low-code platforms . . . 6
3 Analysis of Existing Solutions . . . 7
3.1 Scattered tools . . . 7
3.2 Existing Solutions . . . 7
3.2.1 ERPs . . . 7
3.2.2 Spreadsheet Systems . . . 8
3.2.3 Low-code / no-code platforms . . . 8
3.3 Critique of Exisiting Solutions . . . 9
4 Basic Concepts . . . 9
5 Work Methodology . . . 10
5.1 Agile Methodology . . . 10
5.1.1 SCRUM . . . 10

6 Internship Timeline 10

II Software Specifications **13**

1 Identifying Actors 13
2 Software Requirements Specifications 14
2.1 Functional Requirements 14
2.2 Non-Functional Requirements 14
3 Detailed Software Requirements Specifications 15
3.1 Global use case diagram 15
3.2 Detailed Use Case Description 17
3.2.1 CRUD entity record use case description 17
3.2.2 Launch action/workflow Use Case Description 19
3.2.3 Filter Records Use Case Description 20
3.2.4 Import and Export Records Use Cases Descriptions 21
3.2.5 Manage Notifications Use Case Description 22
3.2.6 Manage Configurations Use Case Description 23

III Design and Architecture **25**

1 Basic Modules 26
1.1 Modules Descriptions 26
1.1.1 Core Module 26
1.1.2 Authentication/Permissions Module 27
1.1.3 Configuration Module 27
1.1.4 Extensions Module 27
1.1.5 Activities/Notifications Module 27
2 Core Module 27
2.1 Global Domain Model 27
2.1.1 Model Structure for Entity-Relationship Models Representation 28
2.1.2 Model Structure for Business Workflows Representation 31
2.2 Source Code in Database 31
3 Authentication/Permissions Module 32
3.1 Authentication 32
3.2 Permissions 33
4 Configuration Module 34
5 Extensions Module 35

5.1 Comments Extension . . . 35
5.2 Attachement Extension . . . 35
5.3 Checklist Extension . . . 36
5.4 Tag Extension . . . 36
6 Activities /Notifications Module . . . 36
7 Single Page Application . . . 37

IV Implementation **39**
1 Technical Architecture . . . 40
1.1 Backend . . . 40
1.2 Frontend . . . 41
1.3 DevOps . . . 43
2 Technologies Used . . . 44
2.1 Backend . . . 44
2.2 Frontend . . . 46
2.3 Other Tools . . . 47
3 User Interface . . . 47
3.1 Configuration . . . 47
3.2 Home Page . . . 48
3.3 Records Page . . . 50
3.3.1 Calendar Mode . . . 51
3.3.2 Reporting Mode . . . 52
3.4 Record Page . . . 53
3.5 Dashboard . . . 55
3.6 Administration . . . 55
3.7 Notifications . . . 57

Conclusion and Future Work **59**

References **60**

List of Figures

I.1 Kostango's logo . . . 3
I.2 Kostango's areas of expertise . . . 4
I.3 Kostango's key partners . . . 5
I.4 Kostango's hierarchical structure . . . 5
I.5 scattered tools . . . 7
I.6 Internship Timeline . . . 11

II.1 global use case . . . 16
II.2 CRUD entity record use case . . . 17
II.3 Launch action/workflow use case . . . 20
II.4 Filter records use case . . . 21
II.5 Import and export records use cases . . . 22
II.6 manage notifications use case . . . 22
II.7 manage configurations use case . . . 23

III.1 Modules dependancy diagram . . . 26
III.2 Core module domain model . . . 28
III.3 Entity-Relationship model example . . . 29
III.4 EAV data example . . . 29
III.5 SCID design . . . 32
III.6 Session-based authentication . . . 33
III.7 Permissions domain model diagram . . . 34
III.8 Extensions module domain model diagram . . . 35
III.9 Notifications module domain model diagram . . . 37
III.10 Request and Response cycle . . . 38

IV.1 Backend Technical Architecture . . . 40
IV.2 Frontend Technical Architecture . . . 42
IV.3 DevOps Lifecycle . . . 43
IV.4 Configuration User Interface . . . 48
IV.5 Home Page . . . 49
IV.6 list of applications . . . 50
IV.7 Records Page list mode . . . 50

IV.8 filters 51
IV.9 calendar week mode 51
IV.10calendar month mode 52
IV.11reporting table 52
IV.12reporting chart 53
IV.13Record Page 54
IV.14Checklist example 54
IV.15Dashboard 55
IV.16Administration entity information 56
IV.17Administration permissions 56
IV.18Notifications 57
IV.19Notification preferences 58

List of Tables

II.1 "create entity record" use case 18
II.2 "consult entity record" use case 18
II.3 "update entity record" use case 19
II.4 "delete entity record" use case 19
II.5 "launch action/workflow" use case 20
II.6 "Filter records" use case 21

List of Acronyms

ERP	Enterprise Resource Planning
CRUD	Create Retrieve Update Delete
UX	User Experience
IT	Information Technology
CRM	Customer Relationship Management
SaaS	Software as a Service
HR	Human Resources
EAV	Entity–Attribute–Value
JSON	JavaScript Object Notation
SCID	Source Code in Database
MVC	Model-View-Controller
ORM	Object–Relational Mapping
REST	Representational State Transfer
URL	Uniform Resource Locator
API	Application Programming Interface
OOP	Object-Oriented Programming
SPA	Single Page Application
AJAX	Asynchronous JavaScript And XML

Abstract

COOs (Chief Operating Officer) in big corporations seek for a global, turnkey, business relevant and agile solution to boost the productivity of Operations. However, business software editors struggle to satisfy COOs's needs as they face 2 dilemmas: **business relevance** vs **globality** and **turnkey** vs **flexibility**. With the current market there is a multitude of overlapping, specialists and unconnected business software with uncovered gaps. All business software also integrates arbitrary choices clients must accept even though their process and organization frequently change.

To solve these problems, we built a global, ultra-flexible and turnkey digital solution that allows us to play around with 3 main axes: Representing any Entity-relationship model, integrating any Business Process workflows and adapting UX to client's needs. Control over these 3 main flexibility axes is transferable across all parties with little to no technical expertise. The solution is also designed to cover a maximum of use cases whilst simplifying and minimizing configuration time. Which results in huge time gains and more control at a fraction of the cost, compared to standard software development methodologies.

Résumé

Les directeurs des opérations des grandes entreprises cherchent toujours une solution globale, clé en main, pertinente et agile pour stimuler la productivité des opérations. Cependant, les entreprises éditeurs de logiciels ont du mal à satisfaire les besoins des directeurs des opérations car ils sont confrontés à 2 dilemmes: **pertinence métier** vs **globalité** et **clé en main** vs **flexibilité**. Avec le marché actuel, il existe une multitude de logiciels d'entreprise en conflit, non connectées, non complémentaires et certains besoins ne sont pas couverts. Tous les logiciels d'entreprise intègrent également des choix arbitraires que les clients doivent accepter même si leur processus et leur organisation changent fréquemment.

Pour résoudre ces problèmes, nous avons développé une solution informatique globale, ultra-flexible et clé en main qui nous permet de jouer avec 3 axes principaux: Représenter n'importe quel modèle entité-association, intégrer tous les workflows métiers et adapter l'UX aux besoins du client. Le contrôle sur ces 3 principaux axes de flexibilité est transférable à toutes les parties avec peu ou sans expertise technique. La solution est également conçue pour couvrir un maximum de cas d'utilisation tout en simplifiant et en minimisant le temps de configuration. Ce qui se traduit par des gains de temps énormes et par plus de contrôle sur les applications pour une fraction du coût par rapport aux méthodologies standards de développement informatique.

General Introduction

Over the last few decades and with the rise of the IT field as a booming industry, the market had been hit with countless digitalization waves [1]. It was therefore flooded with specialist and finely measured software solutions to try and keep up with these waves and touch some of their numerous benefits. This natural and spontaneous reaction created many gaps , inconsistencies and overlaps between the different business software sets that are used within a single company.

Moreover, business specifications rapidly shift and change to match demands and adapt to the needs. Standard software development methodologies cannot always keep up with these changes both in a time and cost effective manner. ERPs (Entreprise Resource Planning) emerged to solve some of these problems. They provide numerous integrated cookie-cutter solutions that are common across all business processes in one unique solution that synchronizes all flows [2]. However, the balance between business scope and business relevance is very hard to pull off especially while trying to keep time and cost to a minimum. With ERPs, all of the integrated solutions are very relevant to the business but that's just about it, flexibility is only limited to the scope themes fixed by ERPs : Sales, Accounting, Human Resources, CRM (Customer Relationship Management), Production, Distribution, e-commerce, Business Intelligence and many others that commonly relevant to most businesses. On the other hand, flexibility is lagging behind when it comes to business needs that are less relevant but very specific. Thus, we are forced to go back to the standard way of building software and use other solutions outside the scopes of ERPs.

It is then conclusive to say that what most businesses need is a solution that maximizes business scope and business relevance. In this project we aim to propose such a solution that will provide businesses the flexibility they need while simultaneously prioritizing business relevance. The approach we propose allows us to represent any entity-relationship model, design business workflows, and adapt the UX accordingly.

The remainder of this report is structured as follows. Chapter 1 presents the context of the project and provides the basic standards and concepts of the project. Chapter 2 details the software specifications. Chapter 3 then illustrates the design and architecture of our solution. Finally, chapter 4 highlights the technical architecture and the implementation of the solution. We'll then conclude our work and present a glimpse of our future work.

Chapter I

Project Context

Plan

1 Presentation of the Host Company 3

1.1 Kostango 3

1.2 Areas of Expertise 3

1.3 Notable Customers 5

1.4 Hierarchical Structure 5

2 Presentation of the Project 6

2.1 Project Definition 6

2.2 Motivations 6

3 Analysis of Existing Solutions 7

3.1 Scattered tools 7

3.2 Existing Solutions 7

3.3 Critique of Exisiting Solutions 9

4 Basic Concepts 9

5 Work Methodology 10

5.1 Agile Methodology 10

6 Internship Timeline 10

Introduction

This first chapter frames the scope of the internship. First, it introduces the host company. Second, it presents the business context of the project and how it was conducted. finally, it illustrates the internship's timeline.

1 Presentation of the Host Company

1.1 Kostango



Figure I.1 – Kostango's logo

Kostango is a company located in Paris and founded in 2015 by two former consultants (Alexandre Johann, Pawel Greim).

Convinced that operational improvement must involve some form of collaboration and the maximum integration of teams across all fields, Kostango offers industrial companies collaborative digital solutions enabling them to carry out improvement initiatives in an efficient and sustainable manner. Kostango favors proximity to its customers in order to fully understand their needs and respond to them as best as possible. As a human-sized and agile company, the Kostango team quickly reacts to provide solutions to its customers. The company constantly seeks to optimize the user experience in order to increase team motivation and contribute to operational improvement.

The company combines industrial and technical expertise in order to contribute to the digital transformation of industrial companies. They work in particular in the aeronautical, automotive and industrial services sectors. Kostango's technical team is specialized in SPA (Single Page Applications) technologies carefully developed in compliance with SaaS (Software as a service) requirements. Each project is carried out with a concern for performance, robustness and security.

1.2 Areas of Expertise

While most of the areas mentioned below are very business specific and does not conform with the generic nature of the proposed solution , they are essential to understand the project's

context as well as give a brief idea of the company's areas of expertise [3] in their previous digital solutions.



Figure I.2 – Kostango's areas of expertise

- **Continuous Improvement**: Also called "Lean management", continuous improvement is inspired by a system successfully implemented in Japanese Toyota factories in the 1970s. And thanks to which factories have become twice as profitable and four times as productive. This continuous improvement approach affects all business processes: financial, administrative and production. It is an operational approach aimed at gradually reducing the malfunctions of a company's processes, customer dissatisfaction or even risks. Focused on creating value and reducing waste, it logically requires neither significant investment nor organizational upheaval likely to disrupt the teams. In return, its effects are only felt in the medium and long term.

- **Cost Reduction**: It is a set of methods to eliminate waste and unnecessary expenses across the entire value chain of a company: production, logistics, marketing, distribution. The Kostango team studies the structure of expenses, and the real needs to detect possibilities for cost optimization; by using a functional analysis in order to detect the weak key factors in costs. The result is an improved profitability and competitiveness of the customer's company in a competitive, unstable and complex ecosystem.

- **Lean Manufacturing**: It is a system that aims to organize a company's production by optimizing its productive resources. It is thus based on a systematic elimination of waste: reduction of stocks, waiting times, waste, defects ... Kostango steps in to quantify the value of the product from the customer's point of view. The goal is to produce only what the customer ordered.

- **Digital Transformation**: The inability to adapt to the ongoing digital revolution, along with the inability to adapt quickly to stay current, can cause many organizations to lose

their relevance. Kostango helps industrial companies to reinvent themselves in the digital world, with digital solutions in all of Kostango's areas of expertise.

1.3 Notable Customers

Kostango works with several well-known groups in the fields of food, transportation and distribution and aerospace.



Figure I.3 – Kostango's key partners

1.4 Hierarchical Structure

The figure I.4 below presents the structure of the company and the different teams within it. Our work was mainly part of the IT team which has 2 interns and 4 developers headed by the chief executive officer Mr. Younes Chaoui. At the time of our internship, the IT team only had 2 interns and 2 developers.



Figure I.4 – Kostango's hierarchical structure

2 Presentation of the Project

In this section we will briefly define the project, its context as well as its motivations.

2.1 Project Definition

The project comes as a natural followup of Kostango's previous lineup of software aimed to digitize industrial companies. It still serves to optimize the operational difficulties encountered by industrial companies. This is the core business and core project of the company since its creation. In their previous software solutions, Kostango understood that any arbitrary choice in the solution harms the flexibility and scalability of the product. So this new project basically offers a solution capable of replacing any business application by:

- Representing any data model.
- Integrating any business workflow.
- Adapting the user interface to the user's choices.

2.2 Motivations

The main motivation behind the development of this project is to provide a solution to the difficulties encountered by industrial companies. These difficulties are a direct repercussion of the rejection of traditional management tools which are often scattered and isolated (excel files, audit grids, flip chart, business software, etc..). The combination of these tools often creates more problems. Therefore, not allowing companies to fully digitize their operations.

2.2.1 Traditional Tools

Using traditional tools like spreadsheet systems or generic task/project management tools (Microsoft Excel , Asana etc..) often offers less business relevance or lacks in modern collaborative features or in the way information is presented.

2.2.2 Low-code platforms

Solutions that are fairly similar to ours already exist on the market but there's a major key difference which evoked the need for our proposed solution.This is explained in full detail in section 3.2.3.

3 Analysis of Existing Solutions

3.1 Scattered tools

To thoroughly manage their resources and processes , companies use numerous scattered tools. For continuous improvement, companies use files generated by platforms such as Asana (online task management tool) shared by email. For cost reduction, Excel files are often used. For the reduction of non-qualities, real physical documents. Sometimes more advanced tools like Microsoft Project or an internally implemented ERP are also used. All communications go by email.



Figure I.5 – scattered tools

3.2 Existing Solutions

Given the generic nature of our proposed solution, allowing it to represent any business case, direct competition is more on specific modules rather than on the entire solution.

3.2.1 ERPs

An ERP is a software package that collects data to optimize the production of the company in different areas: HR, finance, production, etc. ERPs offer content management systems in SaaS

mode. On the other hand, customers seek to manage the business processes and to have areas for improvement to achieve one single goal: to have a more efficient company.

Solutions that customers seek streamlines the distribution of activities and the role of each actor, contrary to ERPs which provide real-time information on stock and resources. Perhaps, the combination of the two is a pragmatic way to steer the business towards digital transformation and continuous improvement. But what is certain is that ERP alone is not sufficient as a solution to the problem we have been dealing with from the beginning.

The first, and perhaps the most striking of the disadvantages, will be the cost. A very contradictory investment for a company looking for a cost reduction platform. Such a solution will involve many costs, whether in terms of prior investment or the training time required to set it up. Its level of complexity is also significant, and often poorly estimated by companies. The figures speak for themselves: 51% of the firms surveyed see the implementation of their ERP as a failure according to a study conducted by the university of Montreal [4].

3.2.2 Spreadsheet Systems

Spreadsheet systems are tailored in a way to allow for analysis and storage of data in tabular form. We differentiate ourselves by offering a unique web-based and modern solution, thus facilitating access to information by employees and allowing "digital" features that Excel does not offer such as notifications or real-time statistics.

3.2.3 Low-code / no-code platforms

Low-code / no-code softwares: these softwares aim to allow customers to create / configure their future application themselves, without needing to code. This is often either a SaaS solution which takes the form of online spreadsheets (Smartsheet, Airtable) or an application creation studio (Zoho Creator). The 2 main obstacles being that customers are looking for turnkey solutions rather than a tool with potentially complicated and unguided configuration. Also this kind of solutions are often too complex because there are too many features related to the power of the configuration for which the responsibility is on the user side. The solution we propose differentiates itself from this type of solution for its ability to deliver a solution that is both turnkey and customizable, the configuration being done on our side and not on the customer's side like low-code software, which allows us to simplify the solution while maintaining its flexibility.

3.3 Critique of Exisiting Solutions

- **High cost**: The high cost of investing in several scattered solutions, in particular that of buying licenses (Excel, asana...) and of the ERP as mentioned in section 3.2.1.
- **Barriers to Information Sharing**: One of the most obvious limitations is the difficulty in sharing content with others. The problem becomes even more delicate when some or all of the information must not be visible.
- **No interactivity**: It is, for instance, not possible to send emails to automatically alert an incident.
- **No automatic synchronization**: Excel files, papers, and other tools are not designed to interact with one another, so employees must manually update all data on all media. No operation should be neglected and should be systematically noted as a potential risk for data integrity.
- **No global vision**: It is generally difficult to have a real time global view of the files/data are shared between the different departments/tools. The time to retrieve each piece of information and compile them to have a complete view, these will have already evolved, the information reported is therefore always out of date. This makes the response time very long in case of an emergency.

4 Basic Concepts

In order to clearly elaborate our report, we first need to explain some idioms in the context of the project.

- **Entity**: An entity is the blueprint of a certain idea/thing. It can be used to create an object. An entity can be assimilated to a class in object oriented programming.
- **Record**: A record is an object for a given entity. It can be assimilated to an object in object oriented programming.
- **Dynamic**: The use of the word **dynamic** in this report refers to the idea that the defining element of a certain software component/notion is determined during **runtime**. For instance, a dynamic database schema, is a schema that is determined during runtime and not during development.
- **Signal**: A signal is simply a trigger that executes some logic if a specific event happens.

5 Work Methodology

The choice of the work methodology is a crucial step since a good approach must adapt to the constraints of the internal and external environment that are imposed on the project.

5.1 Agile Methodology

The agile methodology is a way of managing IT projects that favors dialogue between all stakeholders: customers, users, developers and other project professionals, flexibility during implementation, the ability to modify plans and speed of delivery. It is mainly about breaking away from more traditional practices that are far too rigid and too demanding in terms of specifications. The aim of this methodology is therefore:

- To become more adaptable to unexpected changes.
- To better manage projects, even complex ones.
- To move closer to the customer's expectations since he's involved throughout the production process

Among the agile methodologies we mention Scrum.

Due to the small sized team, we adopted a more generic but custom agile methodology that closely follows the Scrum methodology.

5.1.1 SCRUM

The Scrum methodology was created in 2002, its name is a term borrowed from rugby which means a formation of players [5]. It is based on breaking down projects into iterations also called "sprints". A sprint can have a duration which generally varies between two weeks and one month.

6 Internship Timeline

Our project was carried out according to the timeline presented in the figure I.6 below in the form of a Gantt chart which shows the stages of completion of the project in chronological order as well as the tasks carried out.

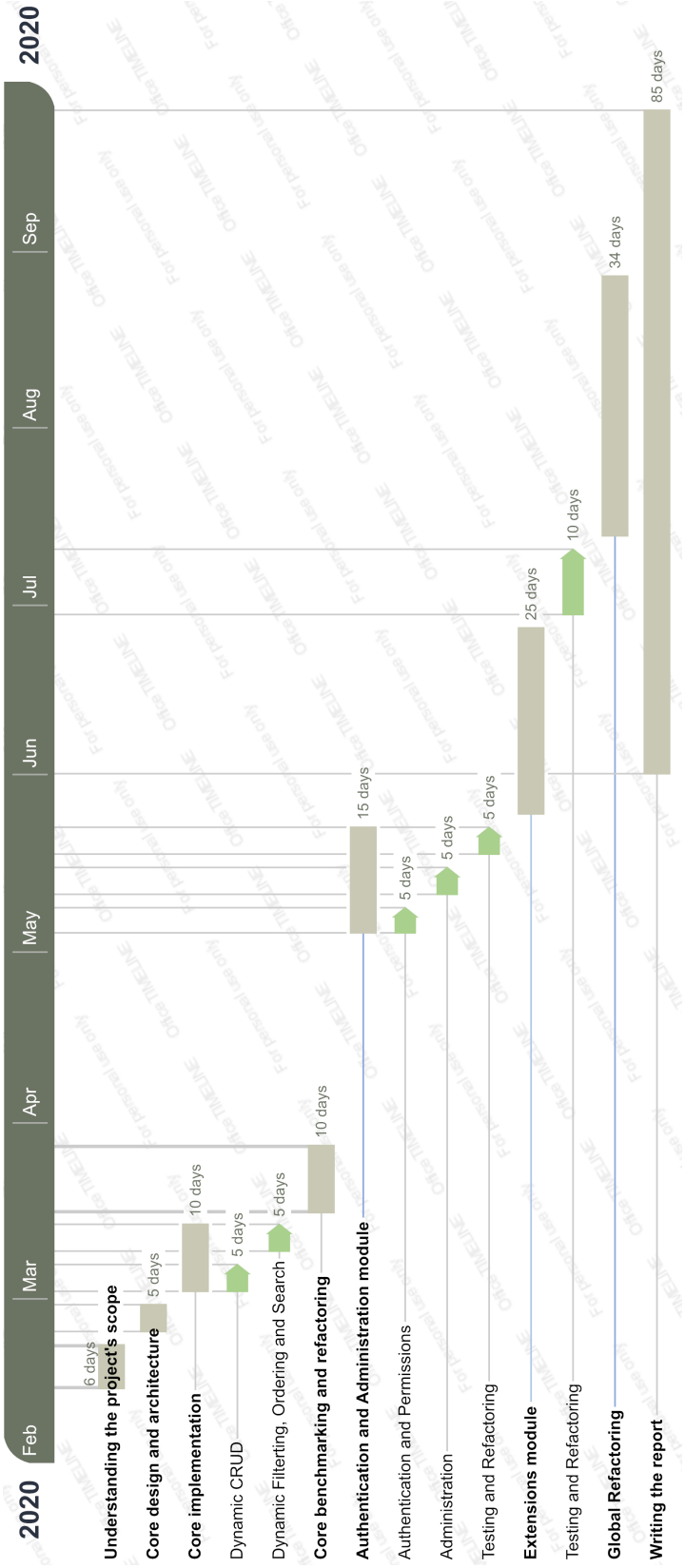


Figure I.6 – Internship Timeline

Conclusion

This first chapter has presented the general context of the project. We first introduced the host company Kostango. We then presented the project , the basic concepts and standards and the schedule established for our work. The following chapter will be reserved for software specifications.

Chapter II

Software Specifications

Plan

1 Identifying Actors 13

2 Software Requirements Specifications 14

2.1 Functional Requirements 14

2.2 Non-Functional Requirements 14

3 Detailed Software Requirements Specifications 15

3.1 Global use case diagram 15

3.2 Detailed Use Case Description 17

Introduction

In this chapter , we will present a complete description of the solution's behavior. We will first identify our main actors. We will then give a general overview of functional and non-functional requirements. Finally, we will go in depth and fully detail our functional software requirements.

1 Identifying Actors

We can distinguish two main actors for our solution:

- **Administrator**: Usually part of the Kostango team and can take care of all the technical/non-technical configurations.
- **Application user**: The user who is meant to use the final configured application. Each user has a dynamic role allowing them to have different roles within the application.

2 Software Requirements Specifications

To successfully complete the project, it is essential to elaborate the functional and non-functional requirements, this will ensure consistency between the client's requirements , those of Kostango, and the architecture to be put in place.

2.1 Functional Requirements

To fully respect our 3 flexibility axes mentioned in section 2.1 of the first chapter ,our application must meet the following functional requirements:

- Represent in full detail any entity-relationship model.
- List all the records in each entity as well as navigate through all the relationships between the records.
- CRUD records for a given entity.
- Filter,group by and order the records for a given entity.
- View records in calendar mode and a pivot table mode.
- Dynamic Dashboards (metrics, charts , tables).
- Import and export records in bulk (csv).
- Launch Actions and follow workflows on records.
- Administrate dynamic permissions.
- Extension related requirements: Comment System, Checklist System, Tags System, Attachments System and a History System.
- All of the above must be covered by a dynamic notification system.
- A configuration module to configure all of the above and much more in full detail.

2.2 Non-Functional Requirements

- **Modularity**: The solution must have a modular architecture; it is essential to have a certain freedom to decide which modules to implement first as well as leave room to easily add new modules.

- **Performance**: Since almost everything is dynamic (data schema and business logic are all defined in the database) we must minimize performance regression compared to a fully static application.
- **Maintainability**: Most SaaS solutions will gradually and continuously change. Whether it's fixing bugs or adding new features; the code must respect the architectural standards put in place. An architecture that will ensure the solution's maintainability while minimizing the work and costs is mandatory.
- **Security**: The platform will cover a multitude of applications which might include ones with sensitive industrial data. We must invest in the protection of this data. Moreover, since our application will have source code in the database, it is essential to protect against code injection attacks.
- **Documentation**: Since we have to keep maintaining it, no IT project is ideal during its first implementation. A documentation will thus become mandatory to better track everything and always have a full understanding of all the moving parts. It also greatly reduces the code base's learning curve.

3 Detailed Software Requirements Specifications

In this section, we will detail the application's behavior for each actor. To do so, we will present the use case diagrams in order to give a global view on the functional behavior of our application and then detail the most prominent use cases.

3.1 Global use case diagram

Below is figure II.1 where we illustrate a global use case diagram for the overall application. It specifies the tasks that can be performed by each actor across the entire application. Users must authenticate in order to perform their tasks. The administrator is the only actor that can thoroughly configure the application and give initial dynamic roles to users which determines which tasks they are able to perform. The administrator is therefore the starting point of all the application, without proper configuration, the application is blank.

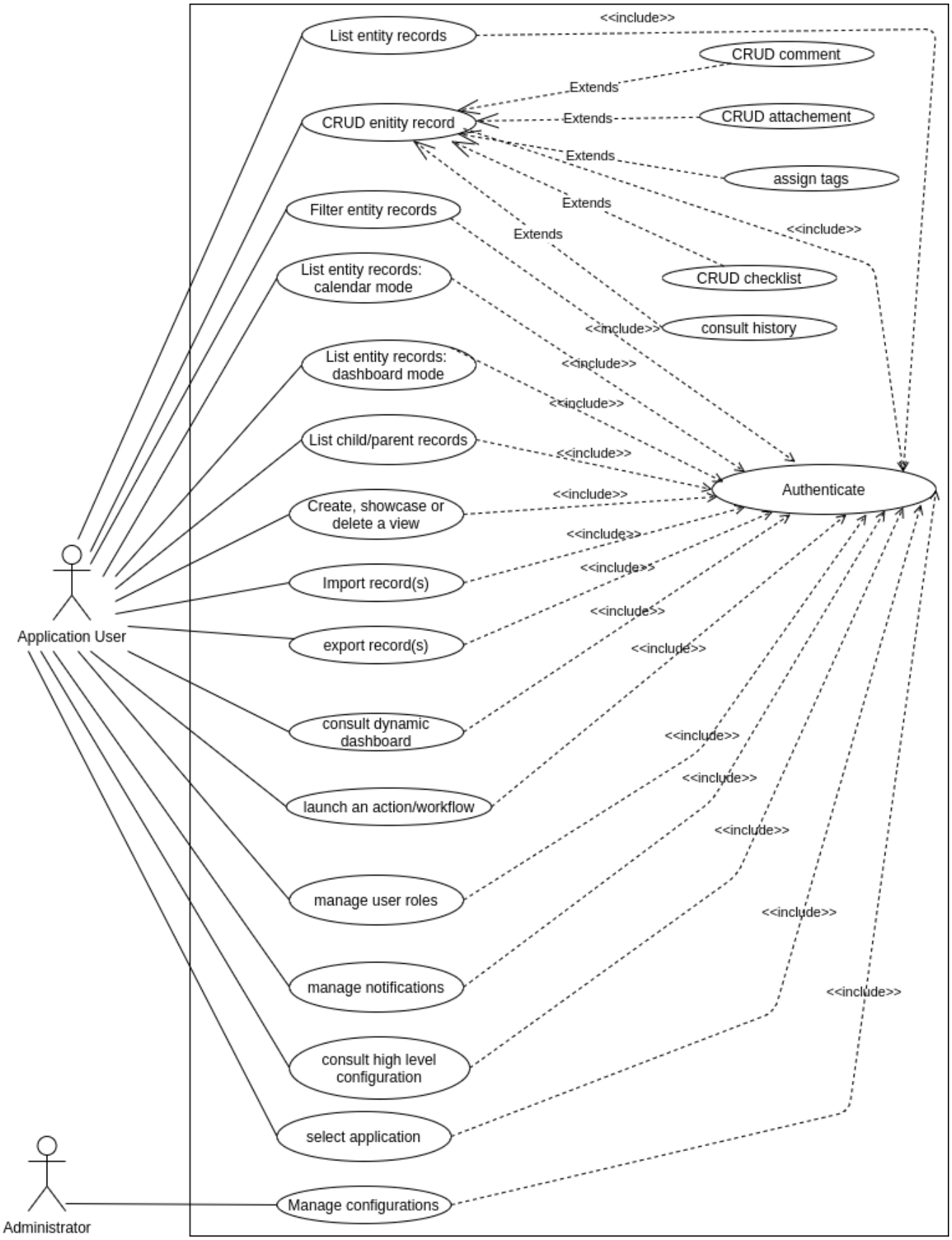


Figure II.1 – global use case

3.2 Detailed Use Case Description

In this section, we will further detail the some of main use cases mentioned above.

3.2.1 CRUD entity record use case description

The diagram presented in figure II.2 details the "CRUD entity record" use case. The use case is divided into 4 main use cases.

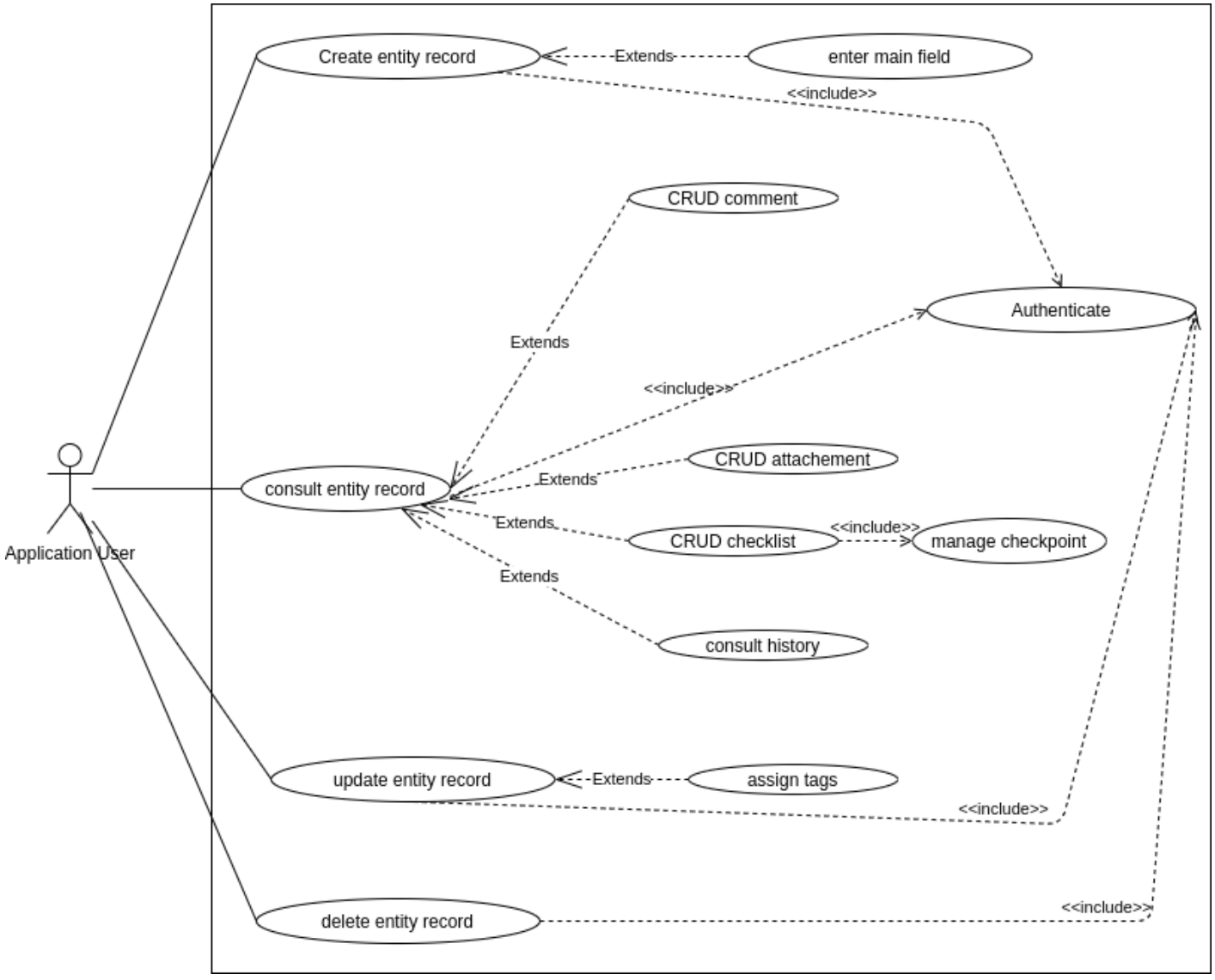


Figure II.2 – CRUD entity record use case

The purpose of the "create entity record" use case is to create a record for a given entity. Creation can be done either from the list mode or directly from the parent entity record where

the child is directly assigned to it. The user can also enter the value for the main field before creating the record. Table II.1 describes the scenario of this use case.

Table II.1 – "create entity record" use case

Use case	Create entity record
Actors	Application User
Pre-conditions	The user is authenticated, has write permissions on the record's entity and is in list mode of a given entity.
Flow of events	• The user goes to the list mode of a certain entity. • He then proceeds to enter the main field if needed and creates the record.
Post-conditions	The system creates the record and launches the associated dynamic signals if they exist.

The purpose of the "consult entity record" use case is to consult a record for a given entity. All the data related to the record (data, comments, attachments etc ..) are optionally managed from that point. The scenario for this use case is illustrated in table II.2.

Table II.2 – "consult entity record" use case

Use case	Consult entity record
Actors	Application User
Pre-conditions	The user is authenticated , has read permissions on that record and is in detail mode of a certain record.
Flow of events	• The user goes to the detail mode of the record in question. • Optionally the user can : – Consult and make basic operations for comments and attachments. – Consult the record's history. – Consult the checklist associated with that record and manage its checkpoints: checkpoint photos management, checkpoint content management, checkpoint status management etc..
Post-conditions	None

The purpose of the "update entity record" use case is to update a record for a given entity. The user can also go ahead and assign or remove tags from the record. The scenario for this use case is presented in table II.3 .

Table II.3 – "update entity record" use case

Use case	Update entity record
Actors	Application User
Pre-conditions	The user is authenticated, has write permissions on that record and is in detail mode of a certain record.
Flow of events	• The user goes to the detail mode of the record in question. • He updates a certain field. • Optionally, the user can reassign record tags.
Post-conditions	The system updates the record and launches the associated dynamic signals if they exist.

The purpose of the "delete entity record" use case is to delete a record for a given entity. We will present in table II.4 the scenario for this use case.

Table II.4 – "delete entity record" use case

Use case	Delete entity record
Actors	Application User
Pre-conditions	The user is authenticated , has the permission to delete the record and is in detail mode of a certain record.
Flow of events	• The user goes to the detail mode of the record in question. • He then deletes the record.
Post-conditions	The system deletes the record and launches the associated dynamic signals.

3.2.2 Launch action/workflow Use Case Description

The diagram presented in figure II.3 illustrates the "launch action/workflow" use case.

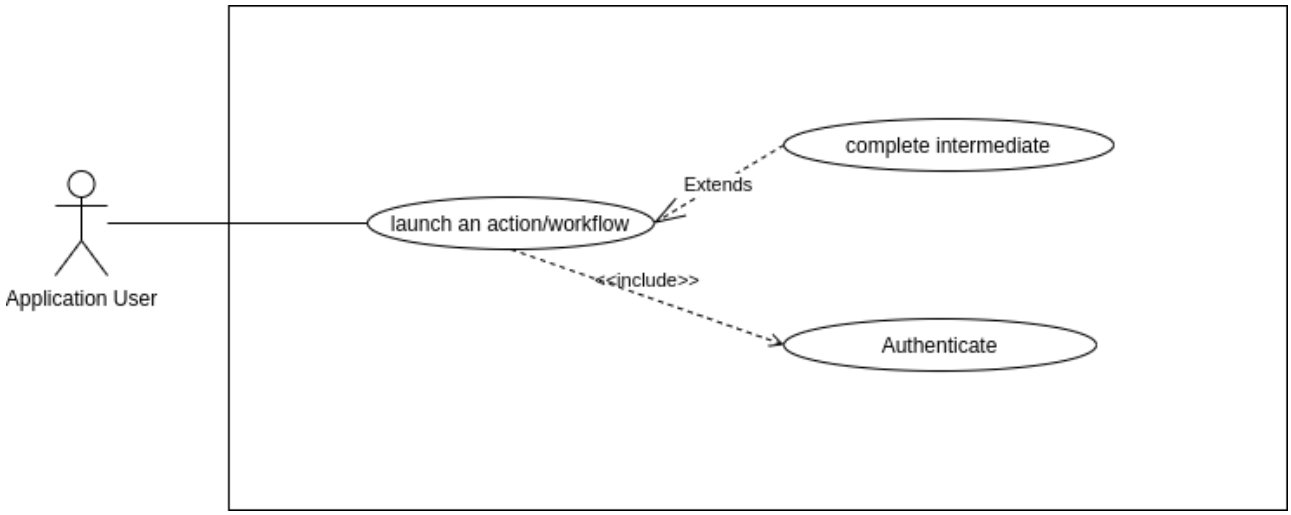


Figure II.3 – Launch action/workflow use case

The “launch action/workflow” use case illustrates the basic scenario of a custom action executed on a record to launch a specific workflow or make changes to any part of the application. The below table II.5 details this use case.

Table II.5 – "launch action/workflow" use case

Use case	launch action/workflow
Actors	Application User
Pre-conditions	The user is authenticated , has write permissions on the record and is in detail mode of a certain record.
Flow of events	• The user goes to the detail mode of the record in question. • He then executes an action. • Depending on the action the user can optionally enter intermediate information to be used by the action.
Post-conditions	The system executes the action and propagates its effects depending on what’s defined in the workflow.

3.2.3 Filter Records Use Case Description

The diagram below in figure II.4 explains the details of the “Filter records” use case shown in the global use case diagram.

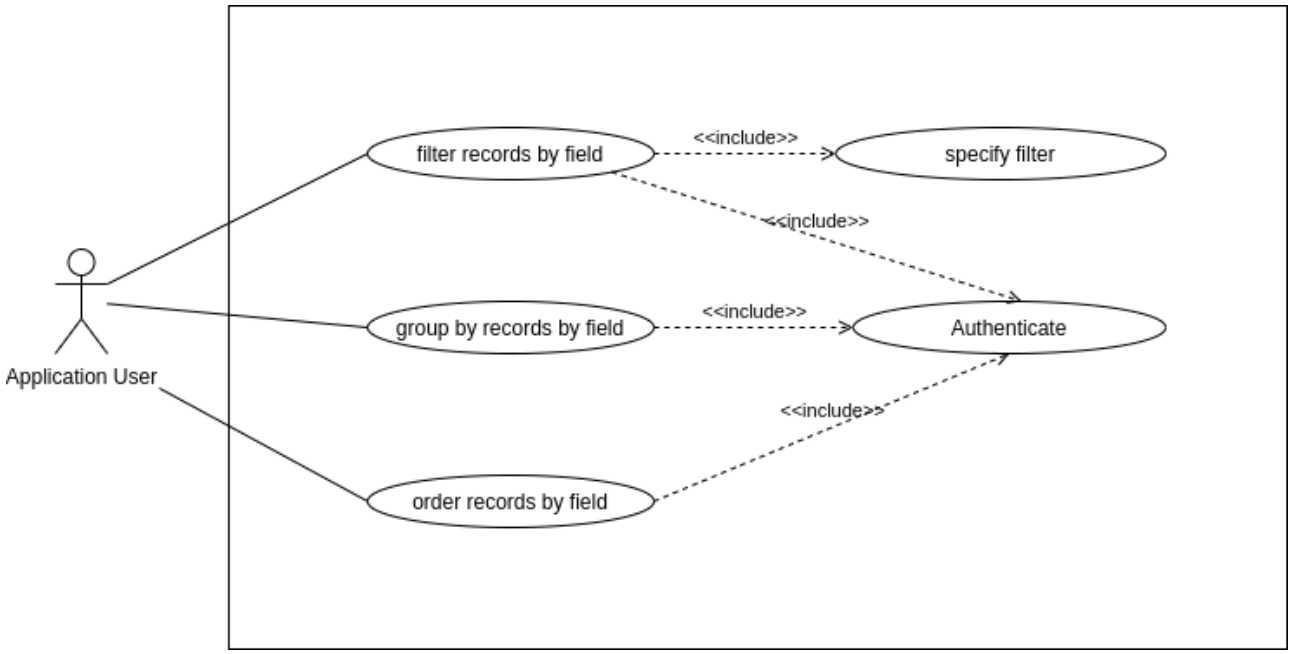


Figure II.4 – Filter records use case

The goal of the "Filter records" is to filter , order and group by records for a given entity and a given field. All of these operations can be combined. We will detail the scenario of this use case in table II.6 shown below.

Table II.6 – "Filter records" use case

Use case	Filter records
Actors	Application User
Pre-conditions	The user is authenticated and is in list mode of a certain entity.
Flow of events	• The user goes to the list mode of the entity in question. • To filter the records the user must specify the filter depending on the field type. • To order records, the user selects the field and the order. • To group records by a certain field, the user selects the field.
Post-conditions	The system executes the filters and shows the results.

3.2.4 Import and Export Records Use Cases Descriptions

The following diagram in figure II.5 illustrates the "Import record(s)" and "Export record(s)" use cases shown in the global use case diagram

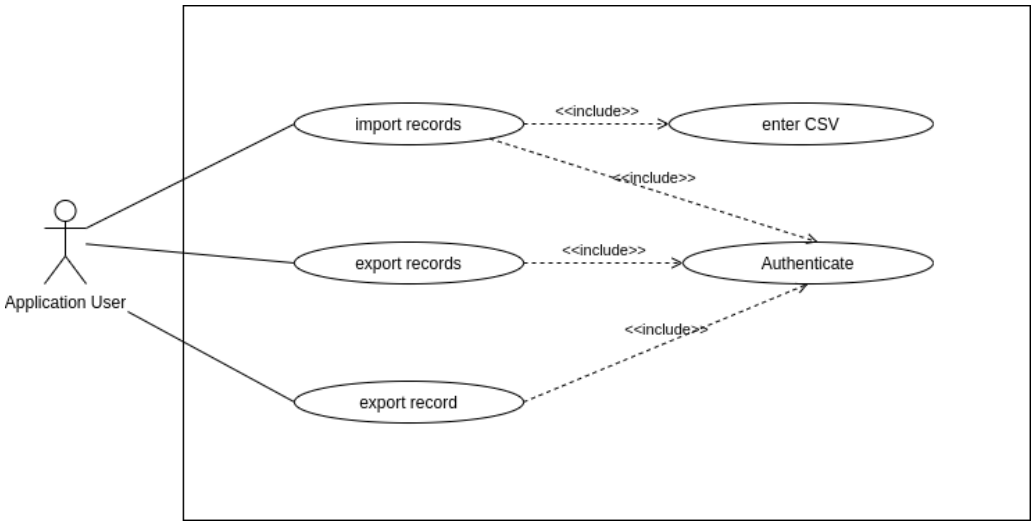


Figure II.5 – Import and export records use cases

The purpose of the "import records" use case is to mainly import records in bulk using CSV data in the adequate form. On the other hand the "export records" use case allows us to export records in bulk in a CSV file. Moreover, The "export record" use case allows us to export a single record in PDF form. The user must be authenticated and must have the right permissions to perform these tasks.

3.2.5 Manage Notifications Use Case Description

Figure II.6 will explain in detail the "manage notifications" use case shown in the global use case diagram.

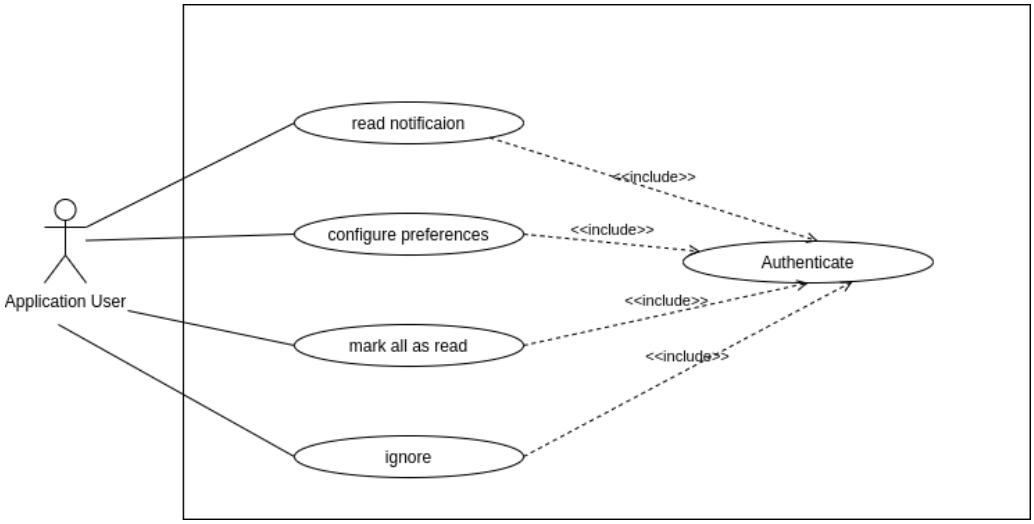


Figure II.6 – manage notifications use case

Upon receival of a new notification, the user can either read or ignore the notification.When multiple notifications are not yet read, the user can mark them all as read in one single action. The user can also configure notification preferences for each entity by specifying the action to be performed (ignore, send notification , send email etc ..) upon triggering a certain operation (addition, deletion, modification, user mentioned etc ..). The user must be authenticated to perform these tasks

3.2.6 Manage Configurations Use Case Description

The diagram presented in figure II.7 illustrates the "manage configurations" use case shown in the global use case diagram.

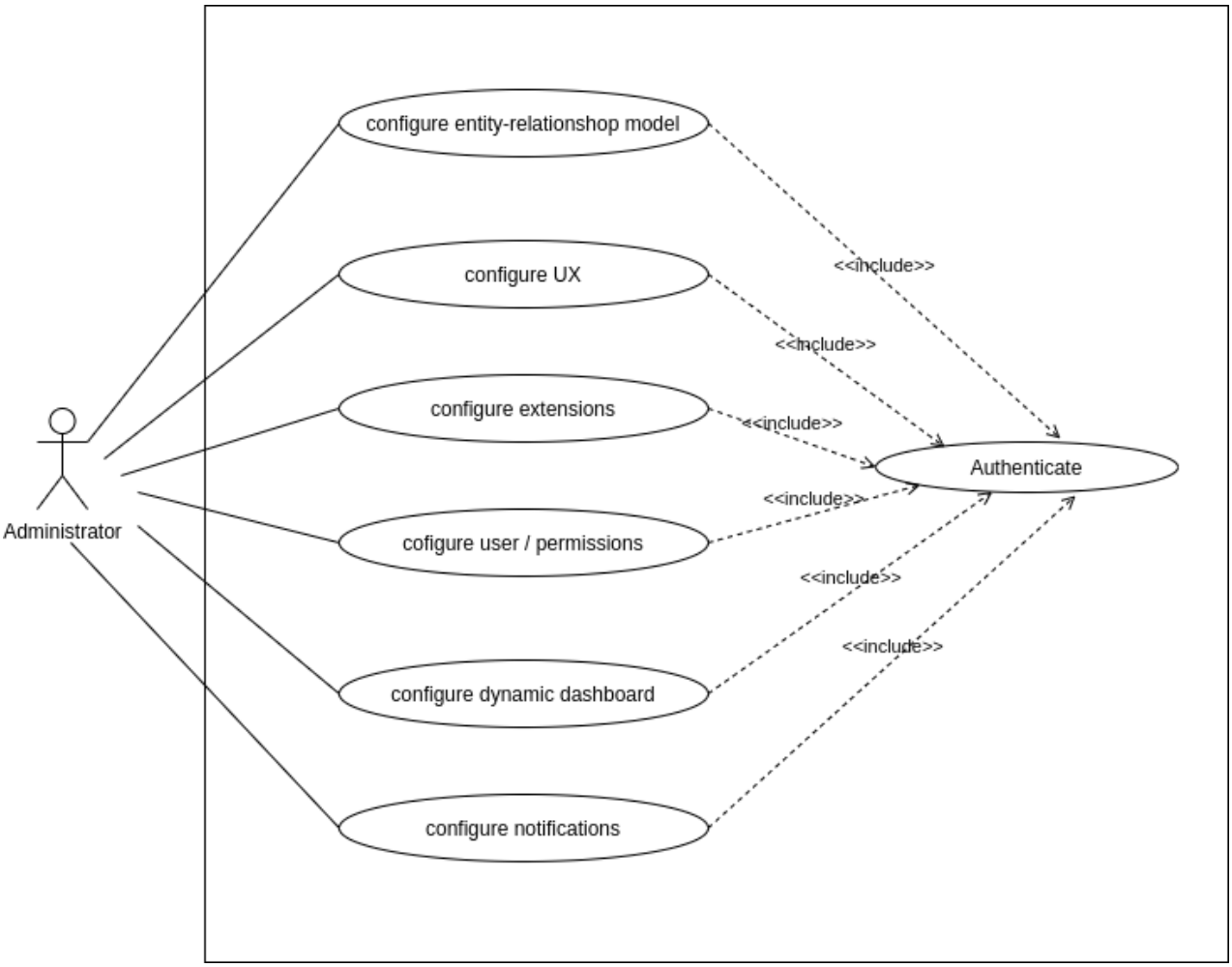


Figure II.7 – manage configurations use case

The "manage configurations" use case describes all the configuration work done by the administrator. Configurations will be mainly done in a simple data entry manner. Configurations

targets the following key areas:

- Entity-relationship model.
- User interface / User Experience.
- Extensions.
- User roles/permissions.
- Dynamic dashboards.
- Notifications.

The user must be authenticated with access to the configuration module before doing any of the configurations.

Conclusion

Throughout this chapter, we have defined the main actors of our system and the requirements to which our solution must be aligned with, through detailed descriptions of the use cases of the different users. The following chapter is devoted to the design and architecture of our solution.

Chapter III

Design and Architecture

Plan

1 Basic Modules 26

1.1 Modules Descriptions 26

2 Core Module 27

2.1 Global Domain Model 27

2.2 Source Code in Database 31

3 Authentication/Permissions Module 32

3.1 Authentication 32

3.2 Permissions 33

4 Configuration Module 34

5 Extensions Module 35

5.1 Comments Extension 35

5.2 Attachement Extension 35

5.3 Checklist Extension 36

5.4 Tag Extension 36

6 Activities /Notifications Module 36

7 Single Page Application 37

Introduction

After identifying the different functionalities of the system in the second chapter, the next crucial step in the software development life cycle is design and architecture. Giving enough importance to this step with the appropriate techniques and tools has proven to be critical for the development of high quality software. To highlight our design, we will start this chapter by giving an overview describing the general architecture of our solution so that we can in a second phase detail our design choices through the appropriate diagrams.

1 Basic Modules

It is essential to push modularization from the very early stages of the software's development process. Having a modular design allows us to see our system as a set of components interacting with each other from the highest abstract view. To properly devide our software into modules, we should consider two main aspects:

- **High Cohesion** between the inner parts of modules.
- **Low coupling** between modules.

The below figure III.1 illustrates the modules for our application.

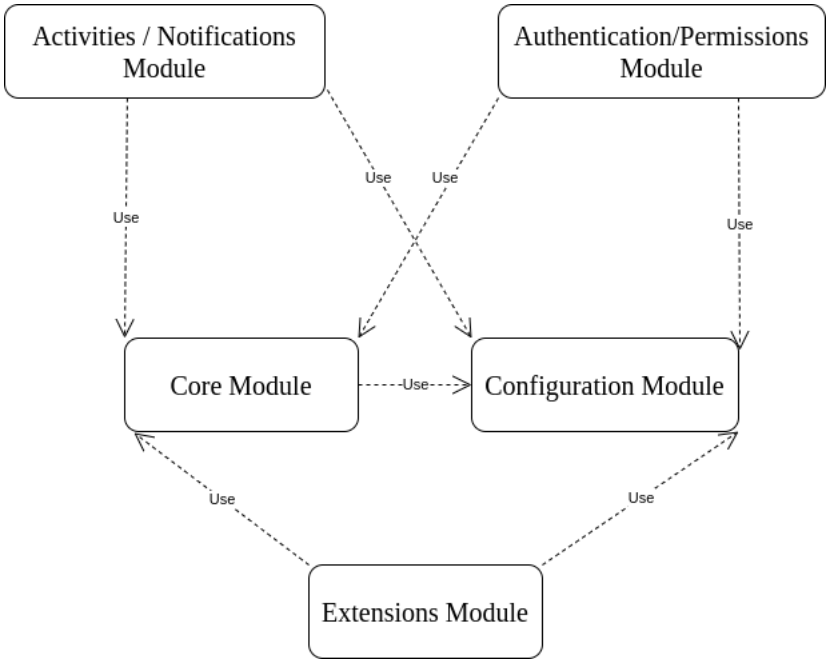


Figure III.1 – Modules dependancy diagram

1.1 Modules Descriptions

1.1.1 Core Module

The core module is the basic module responsible for defining the entity-relationship data model and managing data streams to create dynamic business logic. The end goal is to create the impression of a fully static data model as well as business logic in the eyes of the application user.

1.1.2 Authentication/Permissions Module

The authentication/permissions module is responsible for user and authentication management, it also takes part in managing dynamic permissions.

1.1.3 Configuration Module

The configuration module is a simple and custom administration system that acts as a direct facade to the database in a way to make configuration more simple and accessible.

1.1.4 Extensions Module

The extensions module serves as a way to add extensions to the core functionalities of the application. It greatly enhances the aptitude of creating more business relevant applications. Extensions initially include comments, checklists, tags, history and attachments. The module should allow for more extensions to be added in the future.

1.1.5 Activities/Notifications Module

The activities/notifications module is mainly responsible for tracking all the changes in the application and then managing how they're targeted to the user.

2 Core Module

One of the major specificities in this project that sets it apart from classical business applications is that what's usually defined in the code before runtime is defined in the database and determined at runtime. This is especially important to note as a bad design would have a more noticeable effect on non functional requirements.

2.1 Global Domain Model

One might consider it inefficient to reinvent the wheel and redesign everything from scratch as solutions proposing to represent any entity-relationship model already exist. One of such major solutions is the Entity–attribute–value model (EAV). EAV started as a general-purpose way of representing any form of data structure by using three key components:

- **Entity**: The actual item/table that will be defined.
- **Atrribute**: The parameter or field defined for a certain entity.

- **Value**: The actual data, for each entity-attribute pair.

The below figure III.2 illustrates the domain model diagram for the core module. We will then explain why we haven't adopted the EAV model for entity-relationship models representation.

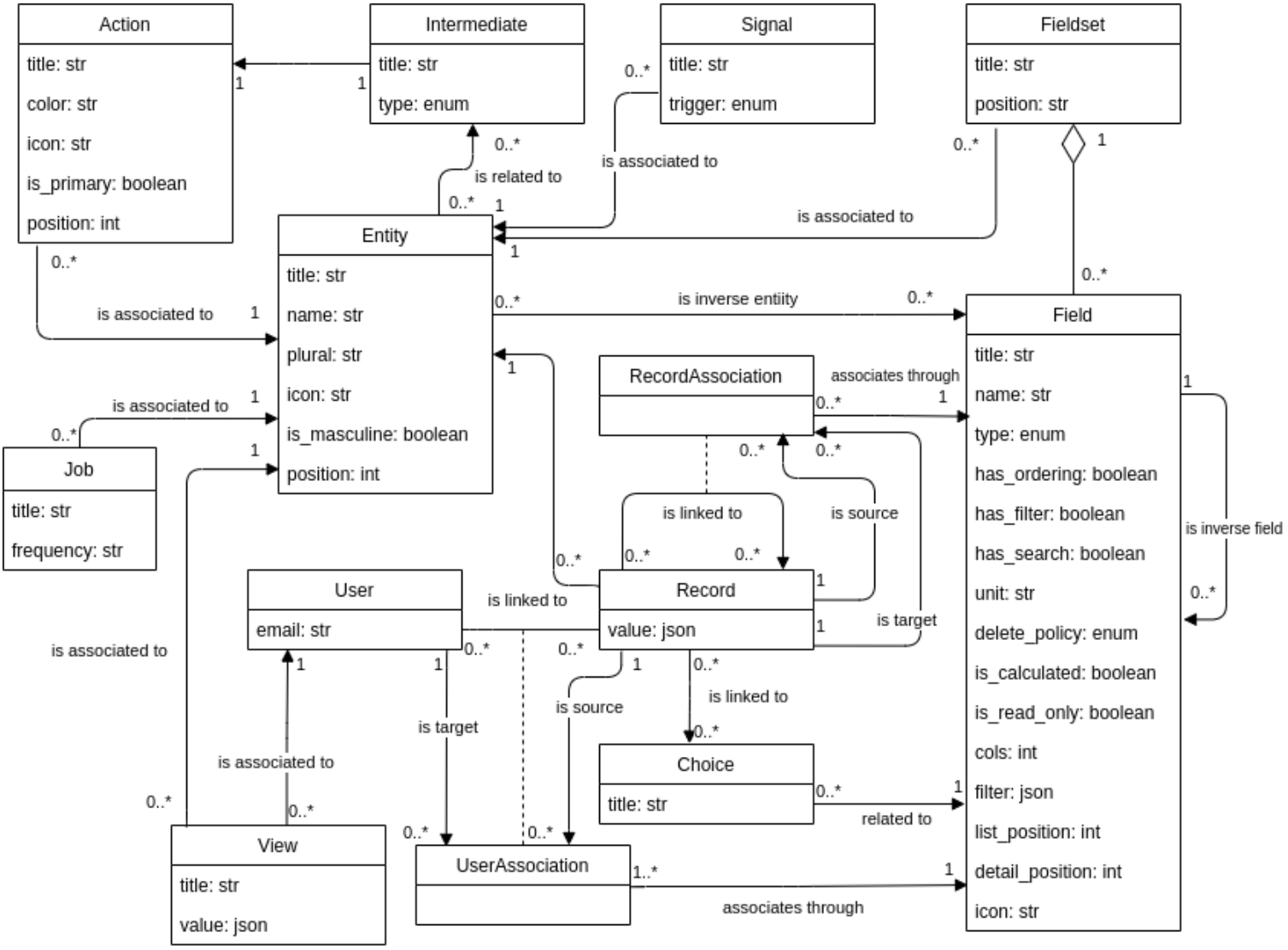


Figure III.2 – Core module domain model

2.1.1 Model Structure for Entity-Relationship Models Representation

2.1.1.1 EAV Analysis and Drawbacks

To better illustrate the model structure we're using and why we haven't adopted the EAV model, let's consider the following scenario presented in figure III.3:

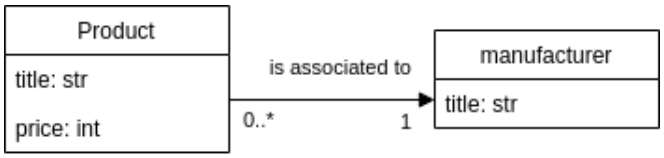


Figure III.3 – Entity-Relationship model example

To represent the above scenario using the EAV model we will need 2 entities **Product** and **Manufacturer**, 3 attributes **title** (Product), **title** (Manufacturer) and **price**. To create a new Product object for example we would need to add 2 entries in the the Value table associating each attribute to a value. To link a product to a manufacturer, we would need to add a new attribute **manufacturer** to the product entity and add a new entry in the Value table associating the attribute to the data ,in this case the ID of the manufacturer.The figure III.4 below illustrates this with detail.

Entity

Product
Manufacturer

Attribute

attribute	entity ref
title	Product
Price	Product
manufacturer	Product
title	Manufacturer

Value

object	entity ref	attribute ref	value
product1	Product	title	abc
product1	Product	price	50
product1	Product	manufacturer	manufacturer1 id
manufacturer1	Manufacturer	title	xyz

Figure III.4 – EAV data example

As we can imagine, this is incredibly slow both when reading or writing specifically because:

1. Simple data structures (integers, strings, dates etc..) are scattered between many entries for the same object.

2. Association fields like foreign keys do not actually represent real links between objects in the database.

Based on the above analysis, we based our design on the need to solve the two problems mentioned above.

2.1.1.2 Proposed Solution

As shown in the domain model diagram in figure IV.2 the "Entity" and "Field" tables are equivalent to the "Entity" and "Attribute" tables in the EAV model. The key difference is that we have a "Record" table instead that represents an entire object for a given entity. To solve the issues mentioned above in the end of the previous section. We propose to do the following:

1. To represent simple data structures, which we defined as follows:

   - Short text.
   - Long text.
   - Number.
   - Date.

   We decided to use a JSON field to store these types. The key value pairs are associated respectively to the field ID and the actual value for that field. Considering a Short text field with an ID of 1 and another Number field with an ID of 2, the values will be stored as follows:

   ```
   {1:'abc', 2:45}
   ```

   Storing all field values for simple data structures for the same record in a single JSON makes database lookups as fast as their static counterparts.

   It is worth noting that short text , long text and numbers are stored as is whereas dates are stored in an iso string format.

2. To represent more complex data structures, which we defined as follows:

   - Single select.
   - User.
   - One to One.
   - Inverse One to One.
   - One to Many.
   - Inverse One to Many.
   - Many to Many.
   - Inverse Many to Many.

   As illustrated in figure IV.2 We decided to use association tables to link:

   - RecordAssociation to link records to other records.
   - UserAssociation to link records to users.
   - Choices are directly linked to records without the need of an association table.

Each association has a source object and target object allowing us to represent all sorts of relationships between objects as well as have a physical link in the database between the linked objects which should drastically improve the performance compared to the EAV model.

It is also worth noting that a lot of automatic work that is usually done automatically by the database such as data integrity checks and applying deletion policies should now be done manually since associations between entities are not static.

2.1.2 Model Structure for Business Workflows Representation

To be able to integrate any business workflows within the application and as illustrated in the domain model diagram in figure IV.2 we make use of the following:

- **Action**: A configurable action that can be executed manually by the user. It is completely flexible since it can be configured by a custom function stored in the database, we will talk more about Source Code in Database in the next section. Actions also make use of **Intermediates** which allow the user to enter more data before launching the action. This data might be useful for the execution of the action.
- **Signal**: Mainly used to execute a function upon triggering some event. The triggers we defined closely follows the CRUD cycle of the record.
- **Job**: Executes a function according to a frequency.

2.2 Source Code in Database

To give full flexibility to our solution, we're taking advantage of a known code manipulation technique called SCID (Source Code in Database). This technique is quite self-explanatory , it is when ready to use code is stored in the database and then later executed at runtime.This allows for a great degree of flexibility that otherwise was not possible [6] as one can simply write a simple function and add it to the database and the application will behave differently.

To make use of this technique, we are going to use configurable function attributes across many models that takes in a predefined function signature that should be respected and returns the adequate result for that attribute. Below is few examples where we use this:

- **Calculated Fields**: A special type of fields where the value depends on other fields, time, or anything else that we define as it's completely flexible.

- **Is active attributes**: These are usually functions that decide whether something should appear or not based on a certain logic.
- **Actions**: Actions are mainly configured via functions too.

We also have plenty of other examples and use cases that should frequently change based on the desired balance between **flexibility** and **simplicity of configuration**.

The below figure III.5 illustrates our design for how SCID integrates with our solution.

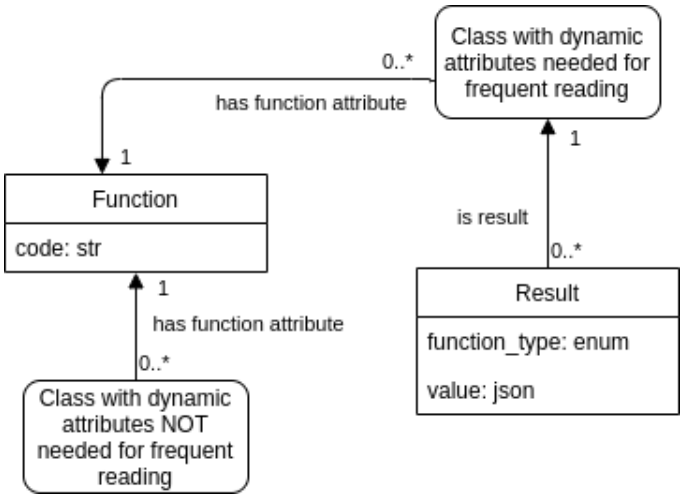


Figure III.5 – SCID design

As illustrated in the figure above we have two kinds of dynamic attributes:

- **Attributes that are needed for frequent reading**: These attributes are calculated whenever any of the factors that they depend on change and are stored in the Result model as shown in the diagram. When they're needed for retrieval, they are not recalculated but rather fetched directly from the Result model.
- **Attributes that are not needed for frequent reading**: Functions for these attributes are only executed at runtime when they're needed.

This division is solely targeted for performance purposes.

3 Authentication/Permissions Module

3.1 Authentication

When it comes to software systems, security is the most important non-functional requirement. Authentication builds the foundation for security. It can be defined as the process of verifying

a person's identity using the required credentials like username and password.

We decided to opt for a classic yet very solid and secure authentication method: **Session-based authentication**. It is based on cookies which makes it stateful. This means that session data must be kept on both the server side and the client side. The server must keep track of active sessions in a database, or in memory if we're not using multiple servers, while a cookie is created on the front-end server that contains a session ID. This session ID is sent with all requests and then compared with session data on the server which in turn processes the request. The figure III.6 below explains this in detail [7].

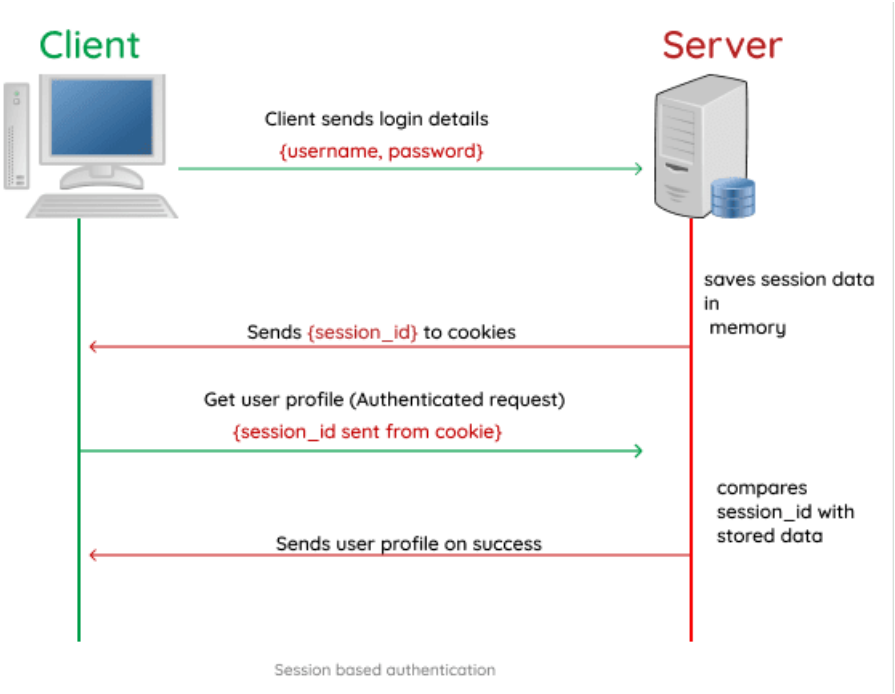


Figure III.6 – Session-based authentication

3.2 Permissions

After successfully authenticating our user. We need to determine the extent to which he's authorized to access the various resources of the application and in which contexts. Since the business entities of our application are dynamic, we need a custom way to authorize users while keeping it flexible and relevant to business logic. In the figure III.7 below, we present the domain model diagram for our permission system.

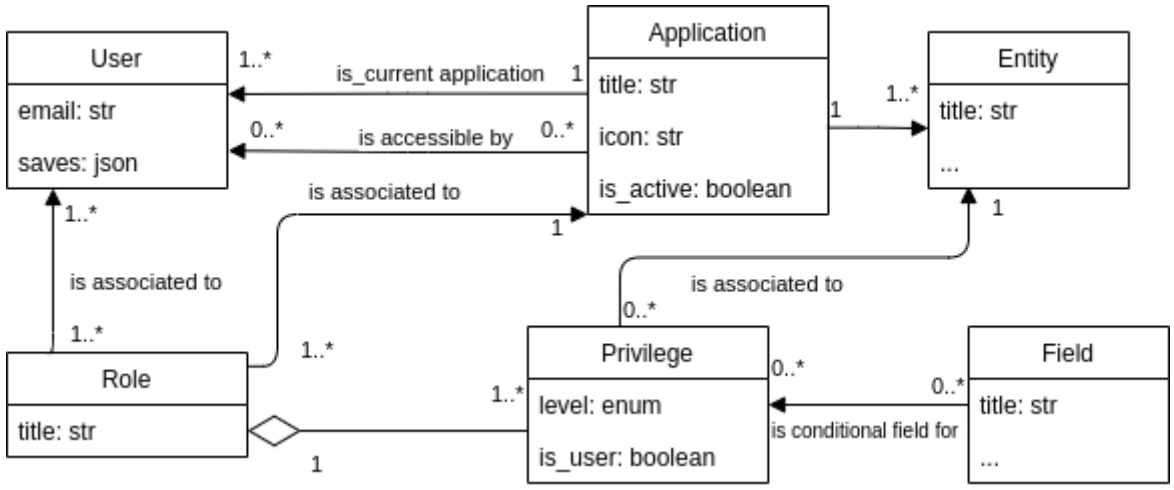


Figure III.7 – Permissions domain model diagram

As illustrated, the simplest form of access control is giving the user access to one or more applications. From that point, we can define permissions for resources within the application itself using the following:

- **Privilege**: As the name implies, this defines a privilege, an atomic access to a certain resource which we defined as follows:
  - Access to a certain entity.
  - Permission to manage users through the "is user" flag.
  - conditional access if the the privilege is linked to a user field and the value for that field matches the authenticated user.

  Each of these privileges can be defined for 3 access levels: Read, write and Administrate.

  These atomic access privileges can later be combined for more specific permissions.
- **Role**: A role is simple a set of privileges. A user can have one or many roles.

4 Configuration Module

The configuration module is the centerpiece that will allow us to configure all the other modules. In our case, providing a simple facade to the database is more than enough to cover all configuration needs. Only minor tweaks to the way data is entered and presented is needed to make the configuration process fast and simple.

5 Extensions Module

As previously explained in the modules description, the extensions module adds business relevance to the application. To better present the design for this module, we'll start by the domain model diagram presented in figure III.8

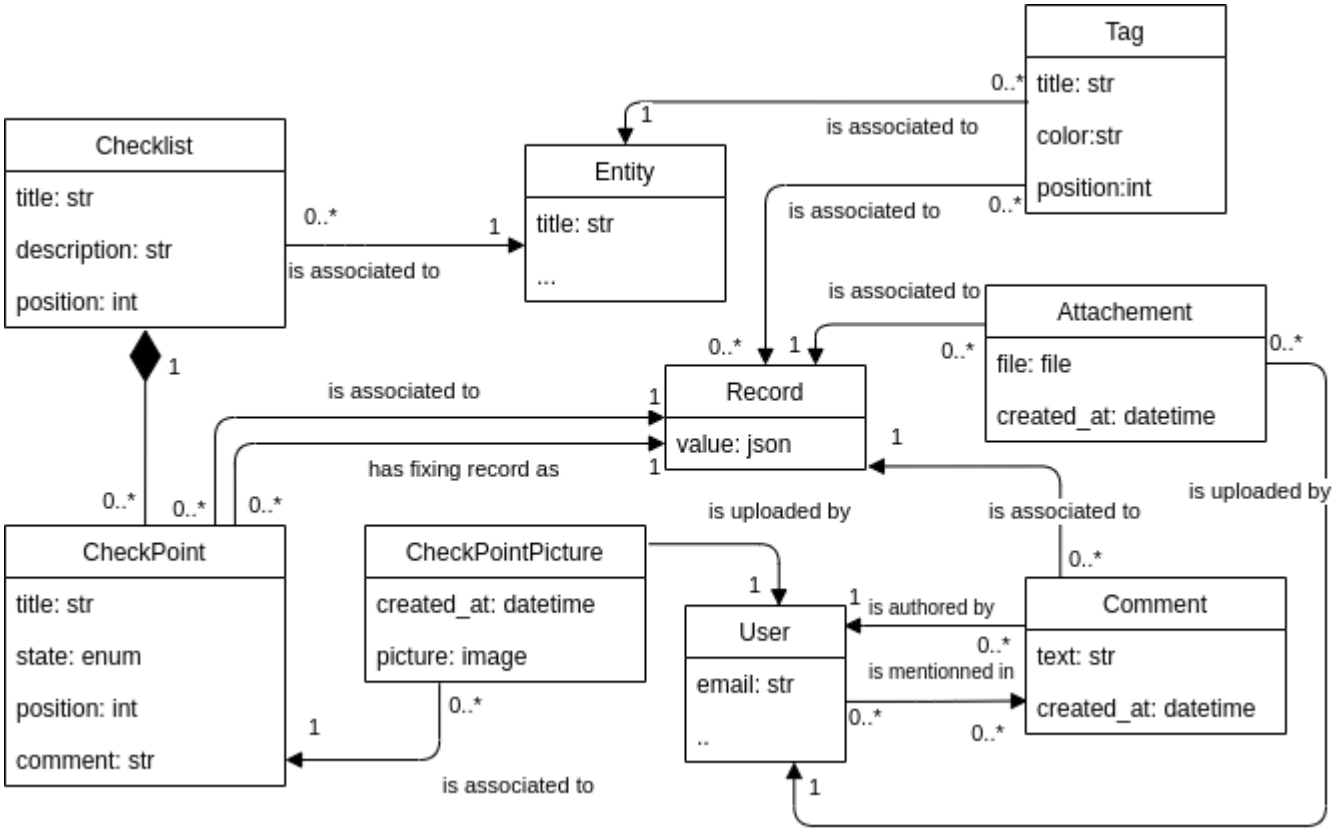


Figure III.8 – Extensions module domain model diagram

5.1 Comments Extension

The comment system is essential to make an application more collaborative. A record can have one or more comments. Each comment is authored by a user and can contain many user mentions.

5.2 Attachement Extension

A record can have one or more file attachments which in turn are uploaded by a user.

5.3 Checklist Extension

A checklist is simply a set of checkpoints and is associated to an entity. A checkpoint can be in one the following states:

- Accepted.
- Rejected.
- Fixed

Each checkpoint can have a comment and set of pictures associated to it. The record fixing the checkpoint is marked as the fixing record.

5.4 Tag Extension

As the name implies the tag system allows us to tag records using custom tags. A tag is associated to an entity and one record can have multiple tags associated to it.

6 Activities /Notifications Module

Figure III.9 below describes the domain model diagram for this module.

In this module we have two main parts:

1. **Activities**: Used to track every change in the application, examples include but are not limited to:
   - Record creation.
   - Record Field update.
   - Action execution.
   - Comment creation.
   - Attachment upload.

   These changes are saved in the snapshot attribute. Activities are not only used to track changes and display them to the user in the history extension but are also useful to troubleshoot certain problems and to trace the changes back to their root.

2. **Notifications**: The notification system simply acts as a way to target activities to users. Notification preferences can be configured for a user and for a given entity, by setting the type (record creation, record update etc..) and the channels (internal , external i.e emails or nowhere). A notification is related to one user and one activity.

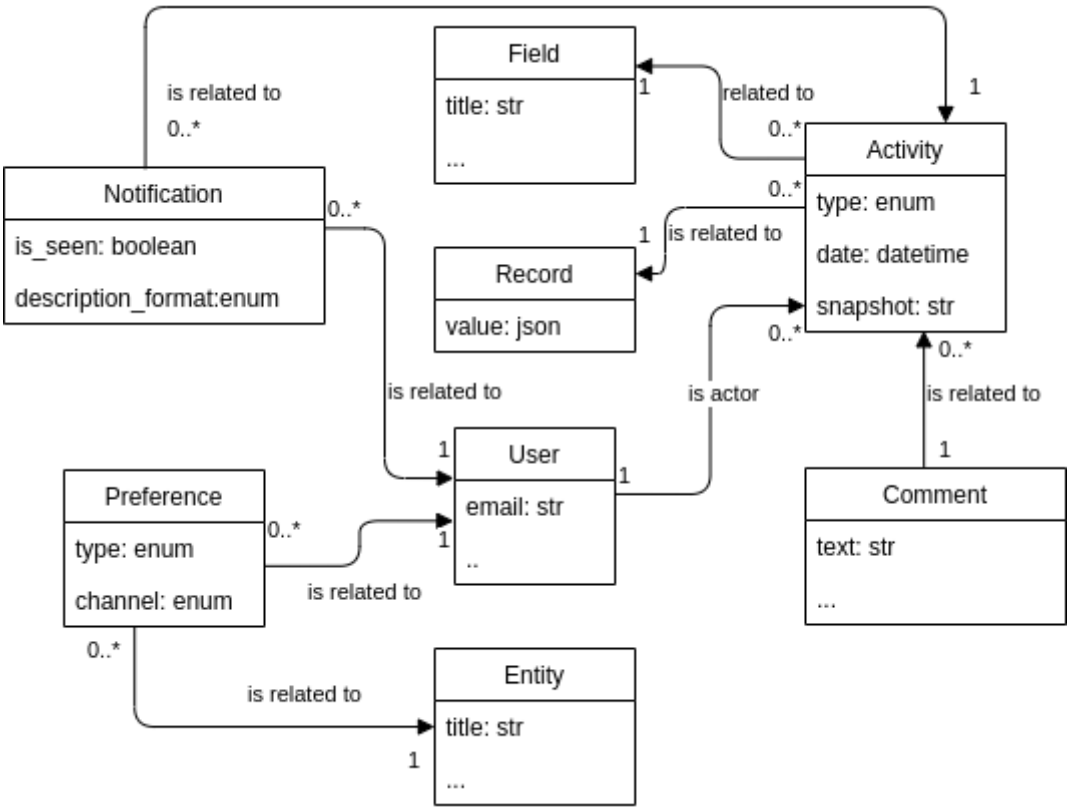


Figure III.9 – Notifications module domain model diagram

7 Single Page Application

It is important to determine the type of web application that we're going to build during the early stages of development. This choice will have a direct impact on the overall architecture of our solution.

We decided to opt for an SPA (Single Page Application). SPAs are web applications that load a single HTML page and dynamically update it as the user interacts with the application. SPAs rely on REST APIs and HTML to create flexible and responsive web applications without having to constantly reload the page. However, this means that much of the work is done on the client side. In a traditional web application, each time a request is sent, the server generates a new HTML page. This triggers a page refresh in the browser. In an SPA, after loading the first page, all interactions with the server take place via AJAX calls. These AJAX calls return data, not HTML, usually in JSON format. The application then uses the JSON data to dynamically update the page without reloading. One main advantage of SPAs is obvious: applications are smoother and more responsive, without the disastrous effect of reloading the page. Another advantage that is perhaps less obvious concerns the way in which we design our application.

Sending application data in JSON format creates a separation between the presentation layer (HTML) and the business logic (AJAX requests and JSON responses). Figure III.10 below illustrates the request and response cycle in our solution.

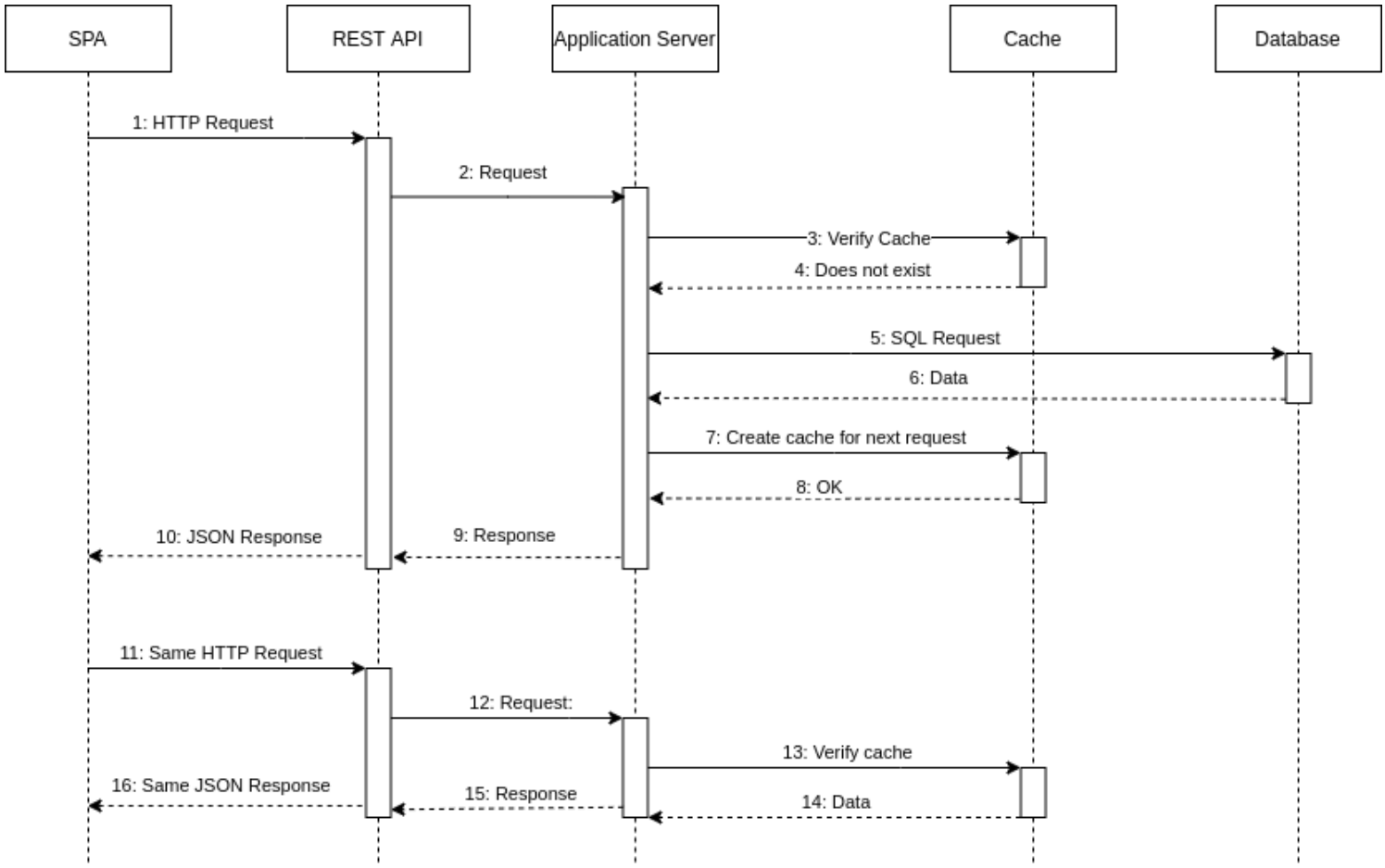


Figure III.10 – Request and Response cycle

Conclusion

During this chapter we explained in full detail the complete design for each module of the solution. We have also fully illustrated our choices to perfectly align with the functional and non-functional requirements. The next chapter will cover the implementation of our solution by detailing the technologies used and providing an overview of the technical architecture. We will also show the end result through a series of screenshots.

Chapter IV

Implementation

Plan

1 Technical Architecture 40

1.1 Backend 40

1.2 Frontend 41

1.3 DevOps 43

2 Technologies Used 44

2.1 Backend 44

2.2 Frontend 46

2.3 Other Tools 47

3 User Interface 47

3.1 Configuration 47

3.2 Home Page 48

3.3 Records Page 50

3.4 Record Page 53

3.5 Dashboard 55

3.6 Administration 55

3.7 Notifications 57

Introduction

In this final chapter, we will present the technical architecture of our solution. We'll then illustrate our choice for the tools and technologies that we have used. Finally, we'll showcase a brief presentation of the user interface through a series of screenshots.

1 Technical Architecture

In this section we will illustrate the technical architecture of the main technologies that we've used to implement our solution which are described in detail in section 2: Django and Angular.

1.1 Backend

For our backend choice, Django, the architecture is presented in figure IV.1.

Generally speaking, it's a model-view-controller(MVC) architecture that divides the application into 3 main components: model, view and controller. In our architecture, the view in Django corresponds to the controller. Django Rest Framework plays the role of the view. The model in our case stays the same. Each one of these components is designed to manage certain aspects of the development of an application. MVC is one of the most common architectural patterns for standard Web development that is used to create scalable and extensible web applications.

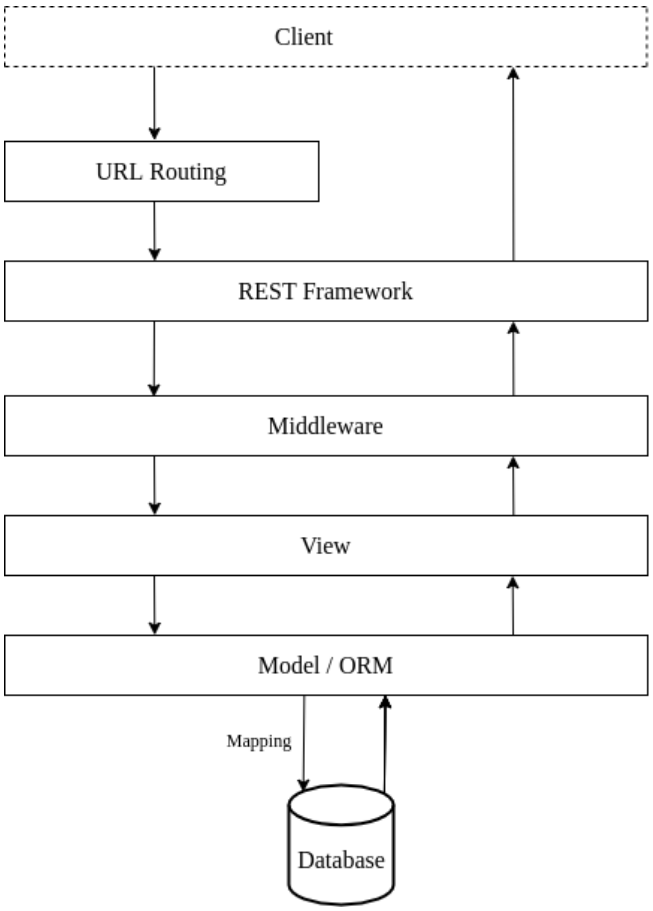


Figure IV.1 – Backend Technical Architecture

- **Model**: A model is the sole and definitive source of information for data. It contains essential fields and any sort of data logic. Usually, each model is mapped to a single database table. Each model is an OOP class [8]. Each attribute of the model represents a database field. Moreover, Django provides a database access API. Once the data models are created, this Database Abstraction API makes it easy to create, retrieve, update and delete objects.
- **View**: The View component in Django is the controller in the MVC architectural pattern. It acts as the interface between the Model and REST Framework components to process all business logic and incoming requests, manipulate data using the Model component, and interact with Middlewares to render the final response.
- **Middleware**: Middlewares are a set of entry points into Django's request and response processing mechanism [8]. It is a lightweight, low-level plug-in system for globally modifying Django's input or output. Each middleware component is responsible for certain specific functions. For example, Django includes a middleware component that associates users with requests using sessions.
- **REST Framework**: Takes care of the serialization of the request and the deserialization of the response. This is the component that manages the "endpoints" of the API.
- **URL Routing**: the URL routing layer is a "mapping" between URL path expressions and controllers (Views in Django) [8].

1.2 Frontend

When it comes to the frontend, we used Angular in conjunction with a state manager, the full architecture is shown below in figure IV.2. The basic blocks of an Angular application are NgModules, which provide compilation context for the components. An Angular application is therefore defined by a set of NgModules. An application always has at least one root module which is the entry point and usually many more feature modules.

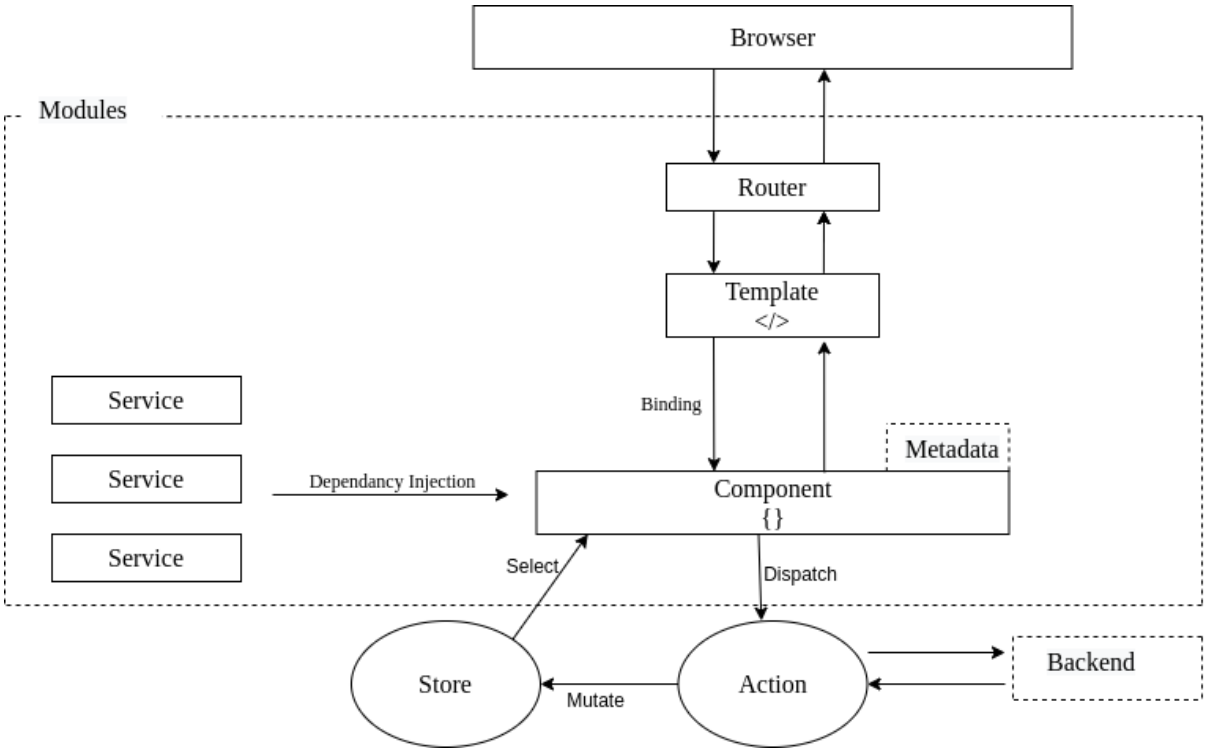


Figure IV.2 – Frontend Technical Architecture

- **Component**: A component is a class, which controls a portion of the user interface [9]. This portion controlled by the component is called a view: a view can be the entire web page, a chat window, a navigation bar, etc. This view is defined in the component's template.
- **Service**:To put it simply, a service allows us to centralize parts of the code that are used by several parts of the application or globally by the entire application. An Angular service can be injected to any Angular class (i.e.: component, Directive, Service or Pipe) using the constructor.
- **Metadata**: Also known as decorators, they are simply used to indicate the role of a class in the application (component, service , directive etc..)[9].
- **Router**: It has the same concept as the URL Routing layer in the backend. However, a slight difference is that for the frontend, it is not desirable to lose all context at each route change. Angular therefore provides a routing system to allow navigation through the different pages of the application.

- **Store**: The store as the name indicates, is a simple JavaScript object that contains the state of the application. The first step in defining the store is to determine its basic elements.
- **Action**: Actions are used to change the state of the store and are usually dispatched from the component.

1.3 DevOps

There are several high level stages in a software project: customer request, development, quality assurance etc... Each step takes a processing period, a queue and the intervention of several actors and therefore less attention towards the customer's needs. We can also note other problems:

- Bugs and technical issues related to dependencies and the difference between the production and the development environment.
- Some modules can take a long time to be tested and deployed.

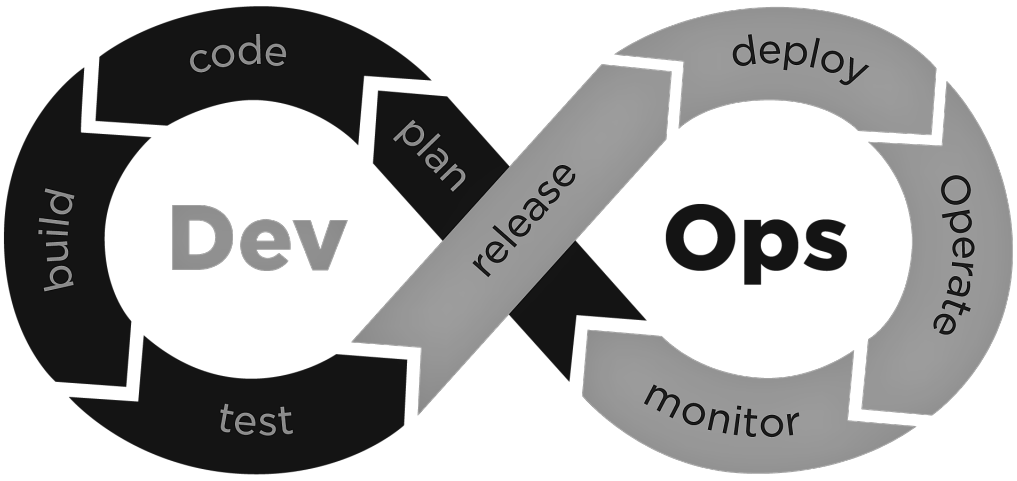


Figure IV.3 – DevOps Lifecycle

The DevOps lifecycle shown in Figure IV.3 above is a work philosophy that aims to align all of the IT staff, under a common goal, it goes beyond agile methodologies towards integrating system operators in work cycles by enhancing communication between different teams to improve the system and ensure continuous delivery to the customer.

There are three different environments. Each environment has its own properties and uses.

- **Development** This is the environment that resides on the developer's machine. This is where we make all the modifications to the source code. The development environment is typically configured differently from the environment in which users use the application. It is connected to a local database or to a dummy database so that everything can be done locally without an impact on real user data. Developers can freely try new features and implement improvements. Many preliminary tests take place in this environment. We can't release the code until we make sure it works at least locally.

- **Staging** The staging environment should be as similar as possible to the production environment. All the source code exists on a server this time instead of a local machine. It connects to as many services as possible without affecting the production environment. All basic testing is done here. All database migrations will be tested here, along with any configuration changes. When performing major version updates, the staging environment also helps to find and resolve any possible problem.

  It's also the environment where we demonstrate to customers to see how things work and to get feedback. An example of such demonstration is the user interface screenshots shown at the end of this chapter.

- **Production** The production environment is where users access the final product after all checks and updates. This is the most important of all environments. This is where the end product resides, and therefore mistakes are fatal. This is why we must first go through the other two environments.

2 Technologies Used

2.1 Backend

- **Python v3.8** Python is an open source programming language created by the programmer Guido van Rossum in 1991. It takes its name from the Monty Python's Flying Circus show [10].

  It is an interpreted programming language, so it does not need to be compiled. An interpreter lets you run Python code on any computer. This allows us to quickly see the results code changes. However, this makes this language slower than a compiled language like C.

As a high-level programming language, Python allows programmers to focus on what they should do rather than how they do it. Thus, writing programs takes less time compared to other programming languages. It is an ideal language for beginners.

- **Django v3.0** Django is a high-level Python web framework that follows the MVC architectural pattern, allowing rapid development of secure and easy to maintain web applications [8]. Created by experienced developers, Django takes care of most of the hassle of web development, so that we can focus on writing our applications without having to reinvent the wheel. It's free, open source, has an active community, good documentation, and several options for free and paid support.

- **Django REST framework v3.11** Django REST Framework is a powerful and flexible toolkit for building REST APIs [11]. The Framework provides serialization that supports both Django and non-Django data sources. It is very customizable and has a comprehensive documentation and excellent community support. Its value is endorsed by international companies including Mozilla and RedHat.

- **PostgresSql v12.3** PostgreSQL is a free, open-source relational database management system with an emphasis on scalability and compliance with technical standards. It is designed to handle a range of workloads, from single machines to data warehouses or web services with many concurrent users [12]. PostgreSQL offers transactions with ACID (Atomicity, Consistency, Isolation, Durability) properties, views that can be updated automatically, materialized views, triggers, foreign keys, and stored procedures. PostgreSQL is developed by PostgreSQL Global Development Group, which is a diverse group of many companies and individual contributors.

- **Unittest v3.7** Unittest is the unittest module of the standard Python library including the unit testing mechanism. The unittest unit testing framework follows the same standards as many major testing frameworks.

- **Memcached v1.5** Memcached is a general purpose distributed memory caching system. It is often used to speed up dynamic websites by caching data and objects in RAM to reduce the number of queries and readings. Memcached is a free and open source software released under the BSD license.

2.2 Frontend

- **Javascript vES6** JavaScript is an interpreted programming language,an object oriented scripting language to be more precise. It is mainly used in web development. It was initially introduced to allow, among other things, introducing small animations or effects on a web or HTML page.

  Created in 1995 by Brendan Eich, JavaScript differs from server side languages by the fact that it can execute on the browser itself, on the user's computer, and not on the web server.

- **Typescript v3.8** TypeScript is a programming language developed by Microsoft in 2012. Its main goal is to improve the productivity of complex application development.

  It is an open source language, developed as a superset of Javascript. What is meant by that is that any code that is valid in JavaScript is also valid in TypeScript.

  However, the language introduces optional features such as typing or object oriented programming. To benefit from these features, no library is required. Just use the TypeScript compilation tool to transpile it (the act of compiling the source code of one language into another language) into Javascript. Thus, the executed code will be a JavaScript equivalent of the compiled TypeScript code.

- **Angular v9.0** Angular is a JavaScript framework that allows us to create Cross Platform web applications: Web, mobile and desktop.

  It is possible to develop applications in Angular either in native Javascript , Dart (a language developed by Google), or TypeScript.

- **RxJs v6.6** Reactive extensions for JavaScript (RxJS) is a library for reactive streams that allows us to work with asynchronous data streams. Reactive programming is a programming paradigm geared towards data flow and propagation of changes.

- **NGXS v3.7** NGXS is a state management pattern + library for Angular. It acts as a single source of truth for your application's state, providing simple rules for predictable state mutations [13].

  NGXS is modeled after the CQRS pattern popularly implemented in libraries like Redux and NgRx but reduces boilerplate by using modern TypeScript features such as classes and decorators.

2.3 Other Tools

- **Ubuntu v18.04** Ubuntu is a free and open source Linux distribution based on Debian. It is one of the most popular operating systems for cloud computing.
- **Docker v18.09** Docker is a tool that can package an application and its dependencies in an isolated container, which can be executed on any server. It's not virtualization, but rather containerization, a lighter form that utilizes certain parts of the host machine for its operation. This approach increases the flexibility and portability of running an application, which will allow it to run reliably and predictably on a wide variety of host machines.
- **Git** Git is a free, open-source distributed version control system. It keeps track of projects and files as they change over time with the help of different contributors.
- **Gitlab** GitLab is a web-based DevOps lifecycle tool that provides a Git-repository manager.

3 User Interface

In this final section, we'll showcase the product through a set of screenshots. The user interfaces will be shown in an order that illustrates the application's entry point and shows the general scenarios and uses case before getting into more specific scenarios.

3.1 Configuration

Configuration is the first and most vital step in using our solution. It is exclusively carried out by Kostango's support team to ensure a proper behavior and a secure application, since we freely allow source code to be added in the database it is vital that this step is done by someone who's familiar with our solution.However, No strong technical knowledge is required. The figure IV.4 shows the user interface for our configuration module, we've utilized django's administration console to have a simple and custom yet effective interface to configure everything.

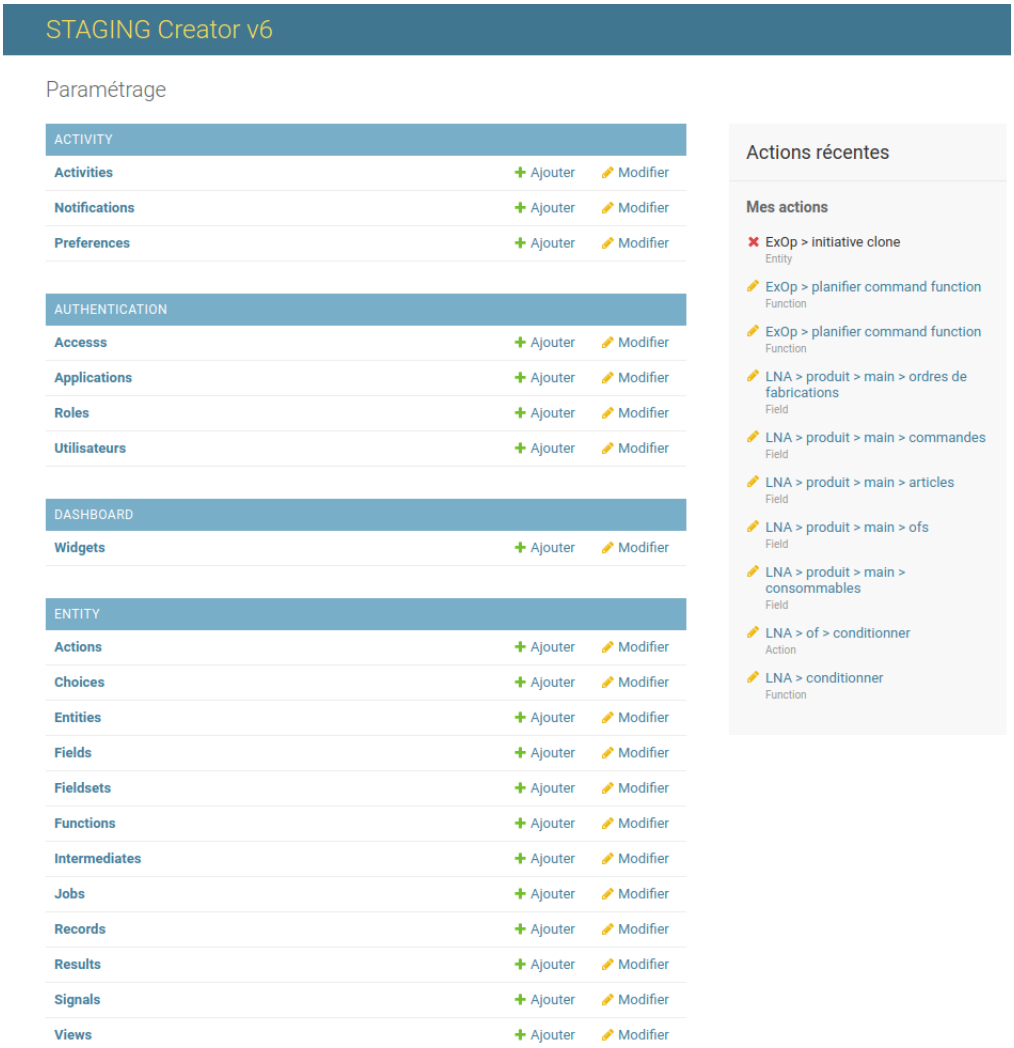


Figure IV.4 – Configuration User Interface

3.2 Home Page

The home page represents a set of views that the user can create in order to be more productive in the application. A view can be a list of records with a specific filter, A calendar, a chart etc.. The home page is illustrated in IV.5. As shown, the view "Incidents de Pascal" is selected. We will also take this chance to detail the components of the Navigation Bar. At the very far left of the navigation bar we can see the different entities of our application. At the very far right there are few quick access buttons for the home page and the dashboard. The figure IV.6 below shows the drop down at the right. As we can see , we have the list of applications that we have access to.

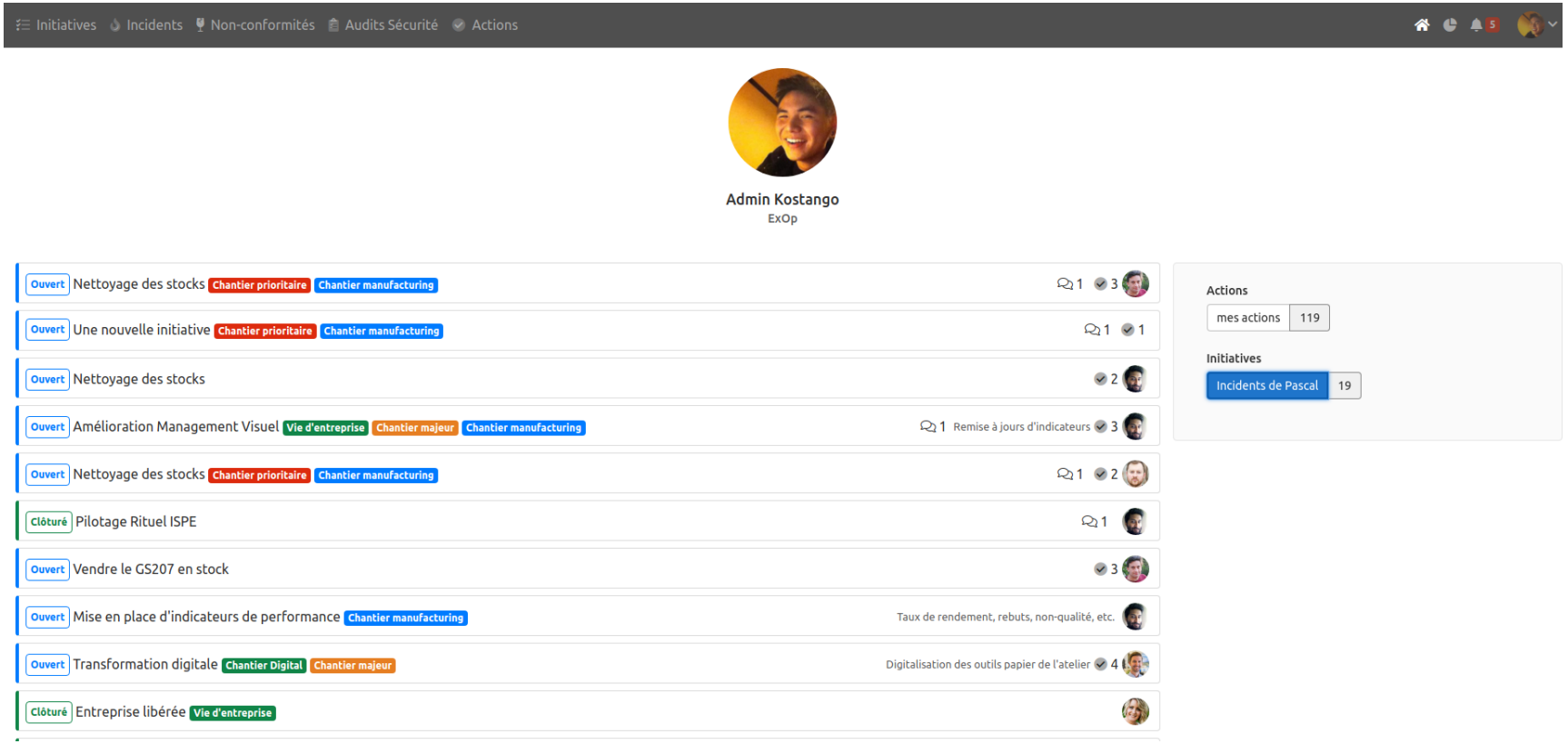


Figure IV.5 – Home Page

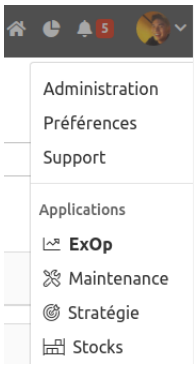


Figure IV.6 – list of applications

3.3 Records Page

The records page for a given entity is supposed to be a global view of that entity. The list mode is showcased in figure IV.7 below.

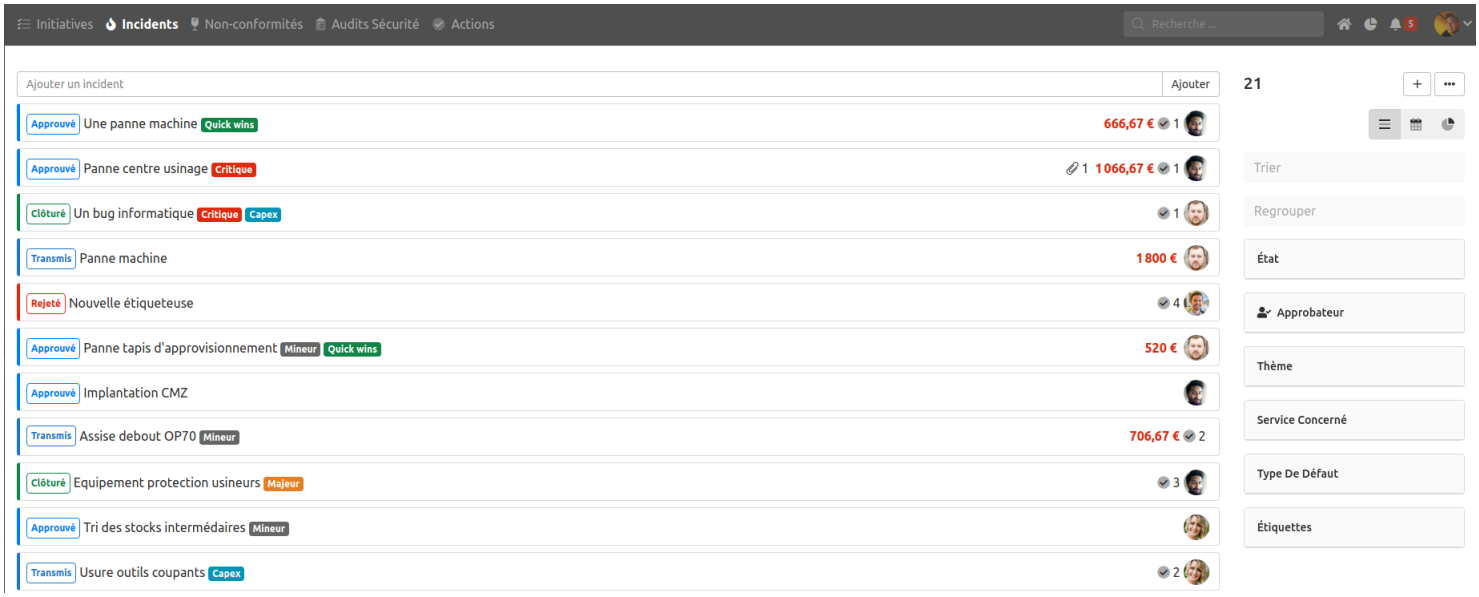


Figure IV.7 – Records Page list mode

We can clearly see a full view of all the records. We can add a new record at the top as well as change the different filters on the right. Figure IV.8 takes us a bit closer to illustrate examples of different filters.

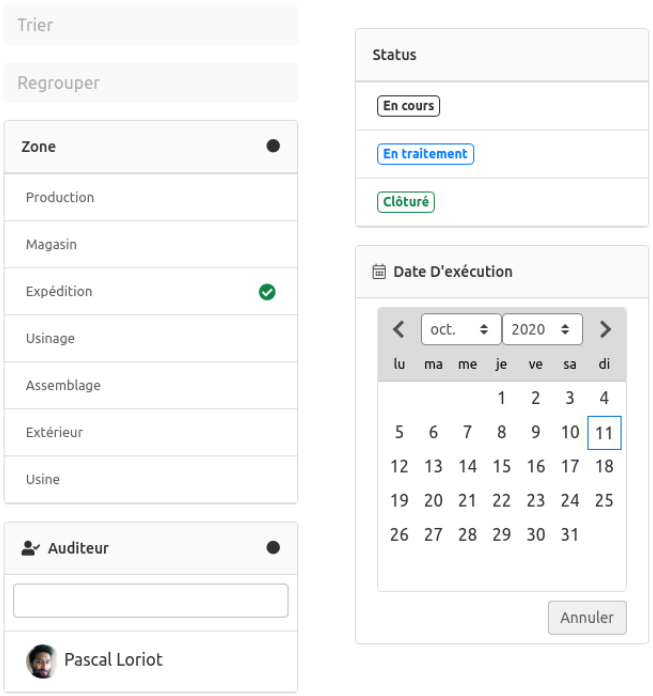


Figure IV.8 – filters

3.3.1 Calendar Mode

The calendar mode relies on a pre-configured date field that acts as a reference point for placing the records inside the calendar. The calendar can be viewed both in week or month mode. Figure IV.9 illustrates the week mode while figure IV.10 illustrates the month mode.

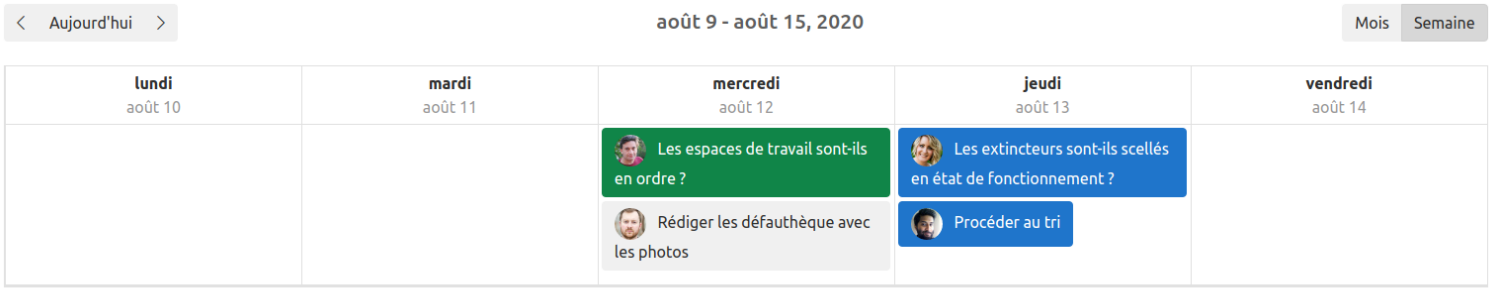


Figure IV.9 – calendar week mode

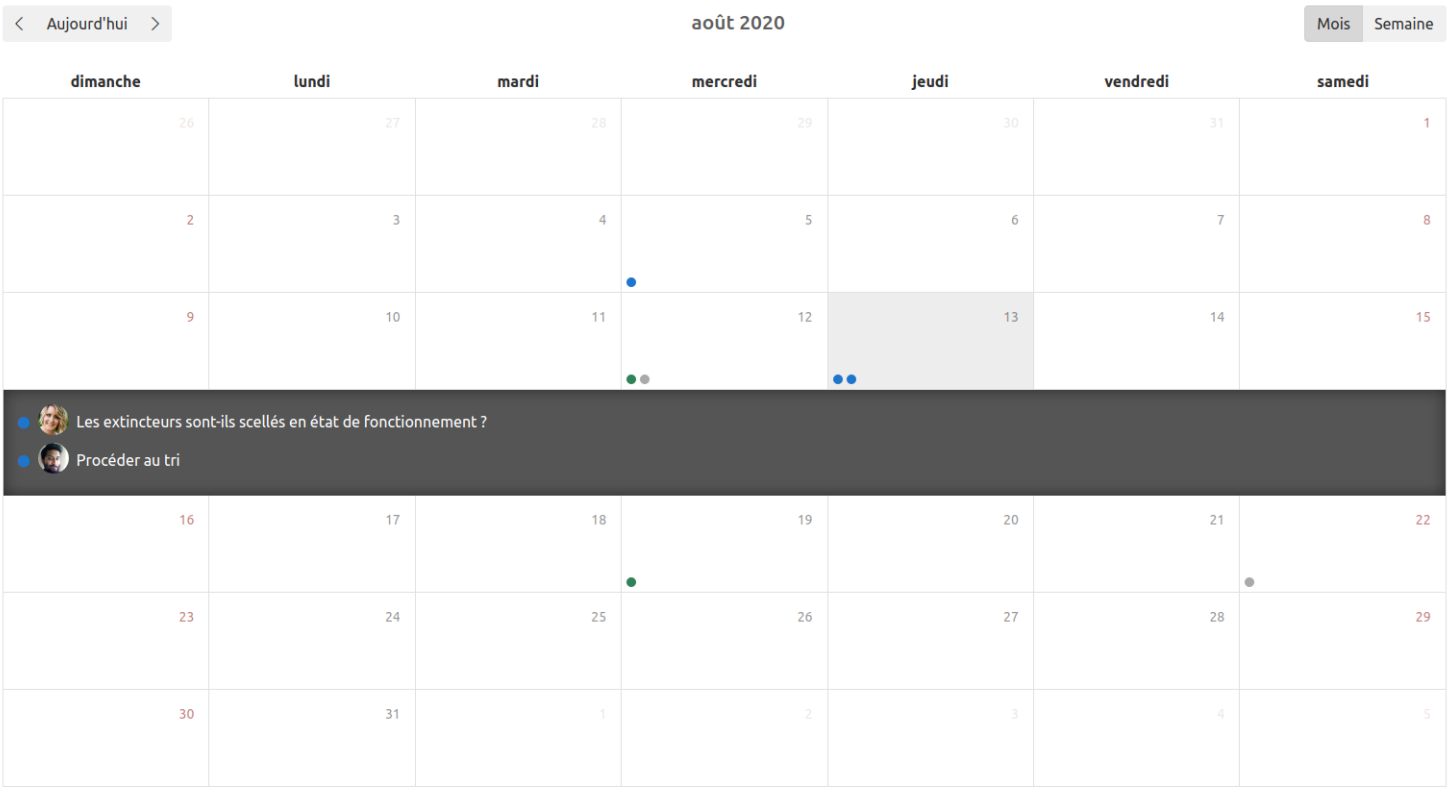
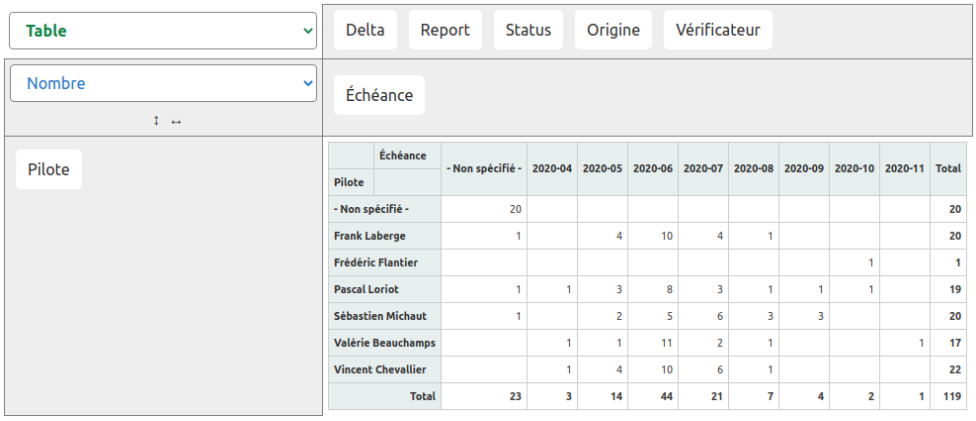


Figure IV.10 – calendar month mode

3.3.2 Reporting Mode

The reporting mode offers a very flexible interface that allows us to represent data in different forms (table, chart etc..). We do this by providing a pivot table and feed it all the data for that entity. Figure IV.11 and IV.12 shows us two examples in the reporting mode.



Pilote \ Échéance	- Non spécifié -	2020-04	2020-05	2020-06	2020-07	2020-08	2020-09	2020-10	2020-11	Total
- Non spécifié -	20									20
Frank Laberge	1		4	10	4	1				20
Frédéric Flantier								1		1
Pascal Loriot	1	1	3	8	3	1	1	1		19
Sébastien Michaut	1		2	5	6	3	3			20
Valérie Beauchamps		1	1	11	2	1			1	17
Vincent Chevallier		1	4	10	6	1				22
Total	23	3	14	44	21	7	4	2	1	119

Figure IV.11 – reporting table

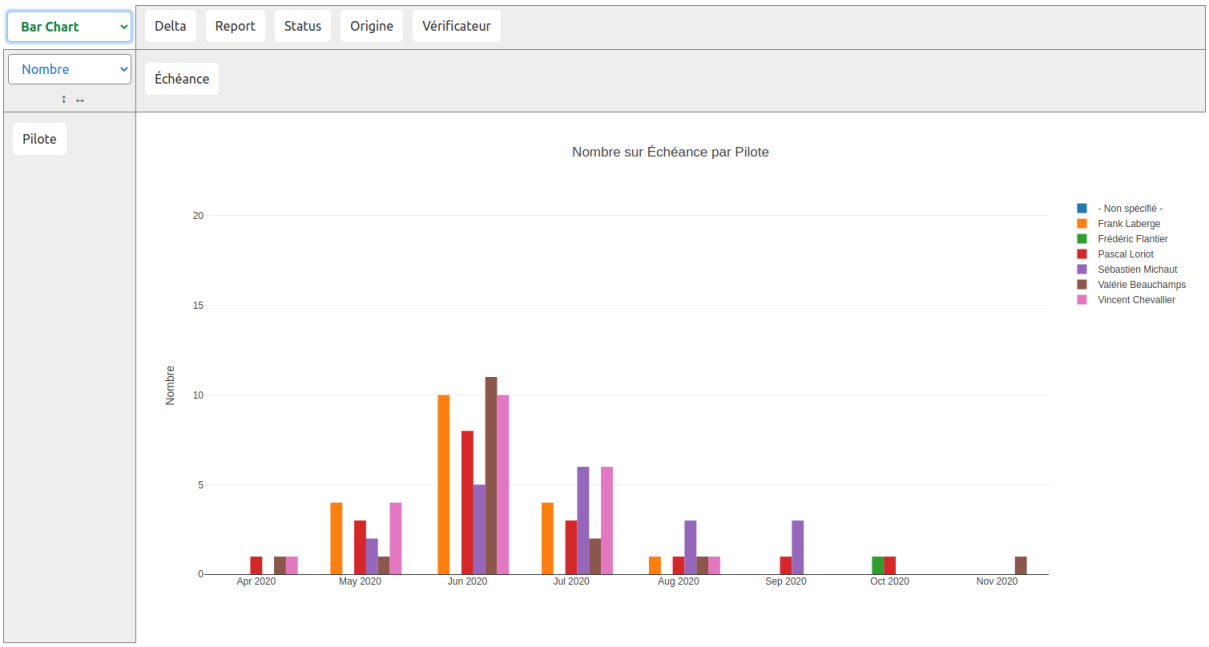


Figure IV.12 – reporting chart

3.4 Record Page

The record page represents a closer look on all the details of one record. As illustrated in figure IV.13, this is where we can do basic operations on the record (read, update, assign tags, execute workflows etc..). On the bottom of the page we can also see related objects and do extension related tasks.

From the related objects windows (Actions in the shown example), we can create and associate more objects to the record. It is also possible to export the page in PDF.

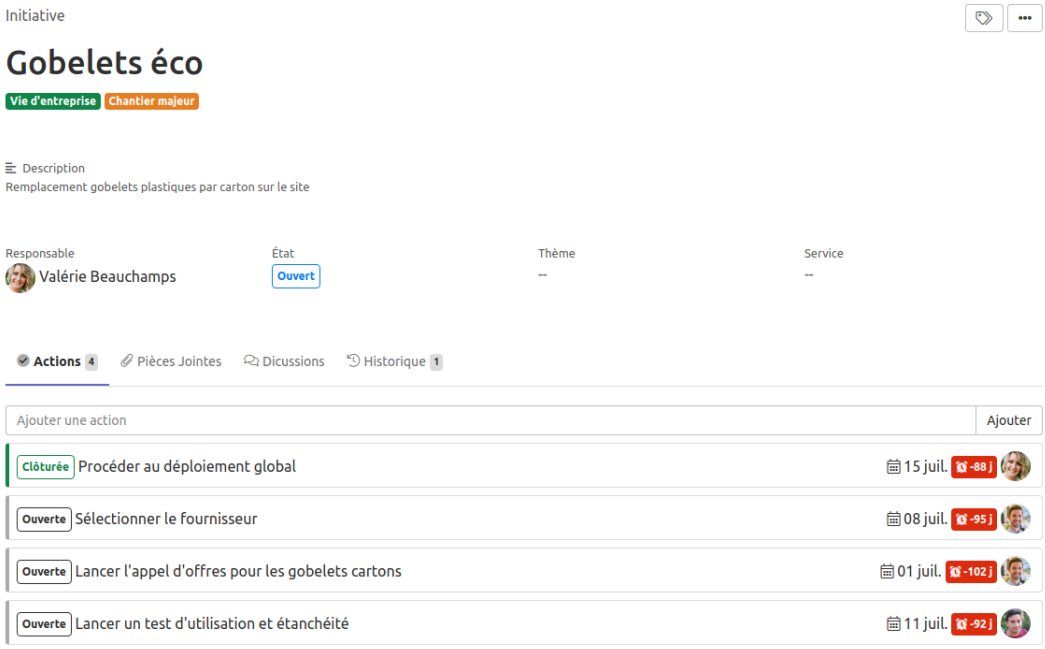


Figure IV.13 – Record Page

Let's take the checklist extension as an example. The figure IV.14 shows a checklist example.

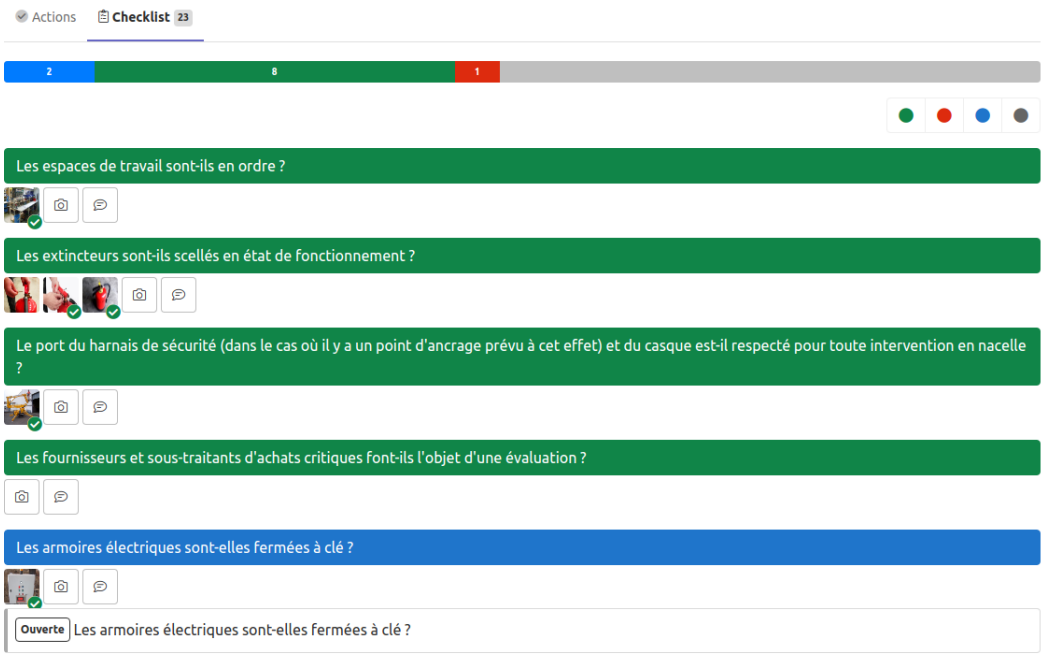


Figure IV.14 – Checklist example

3.5 Dashboard

The reporting mode in records page is not enough to cover all reporting needs since it limits the data to that of one entity. So there's a possibility for other reporting elements to be configured and added in a dashboard. Configuration is done by mainly providing a data function as the well the type of the reporting element (bar chart ,metric, table etc..) The figure showcases an example of a configured dashboard.

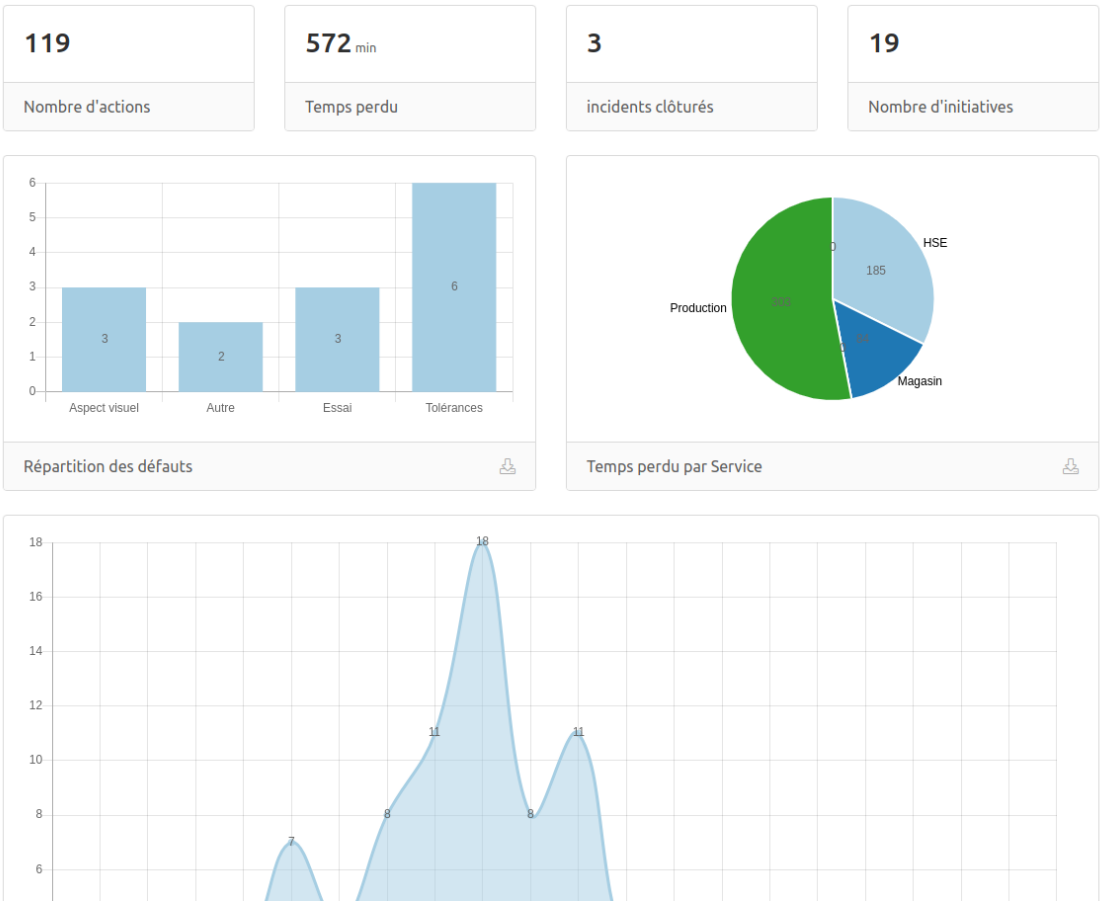


Figure IV.15 – Dashboard

3.6 Administration

The administration section serves two main purposes:

- Showing basic information about each entity: Fields, Actions, Signals etc ..
- Managing users and permissions

This is illustrated in figure IV.16 and IV.17

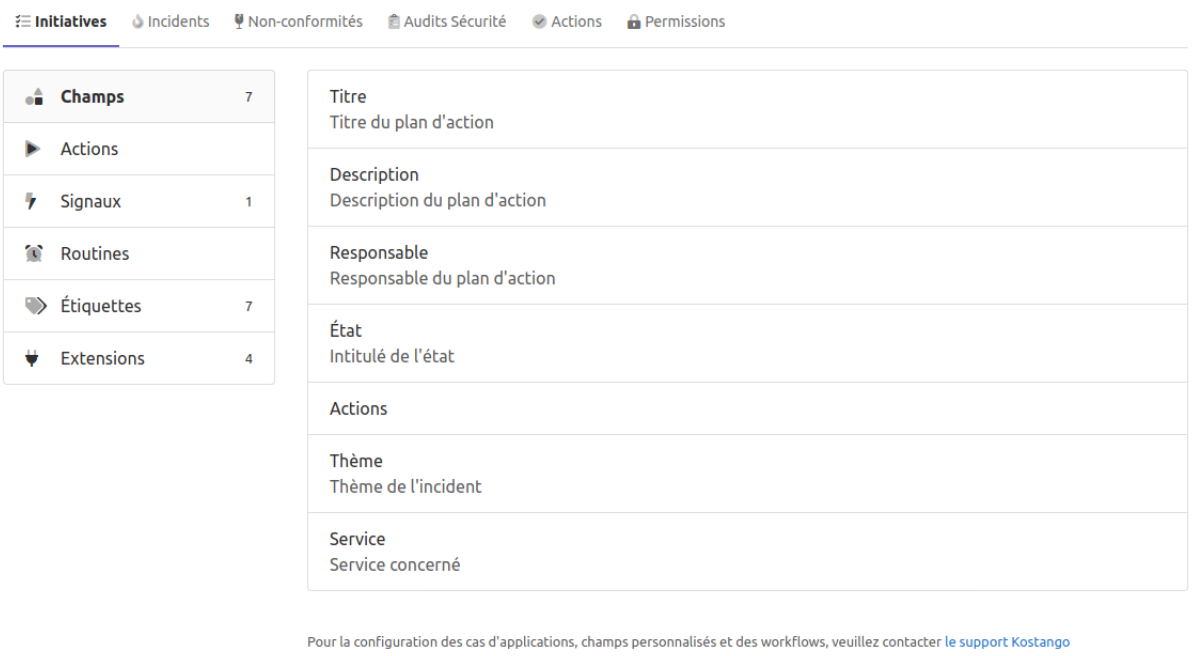


Figure IV.16 – Administration entity information

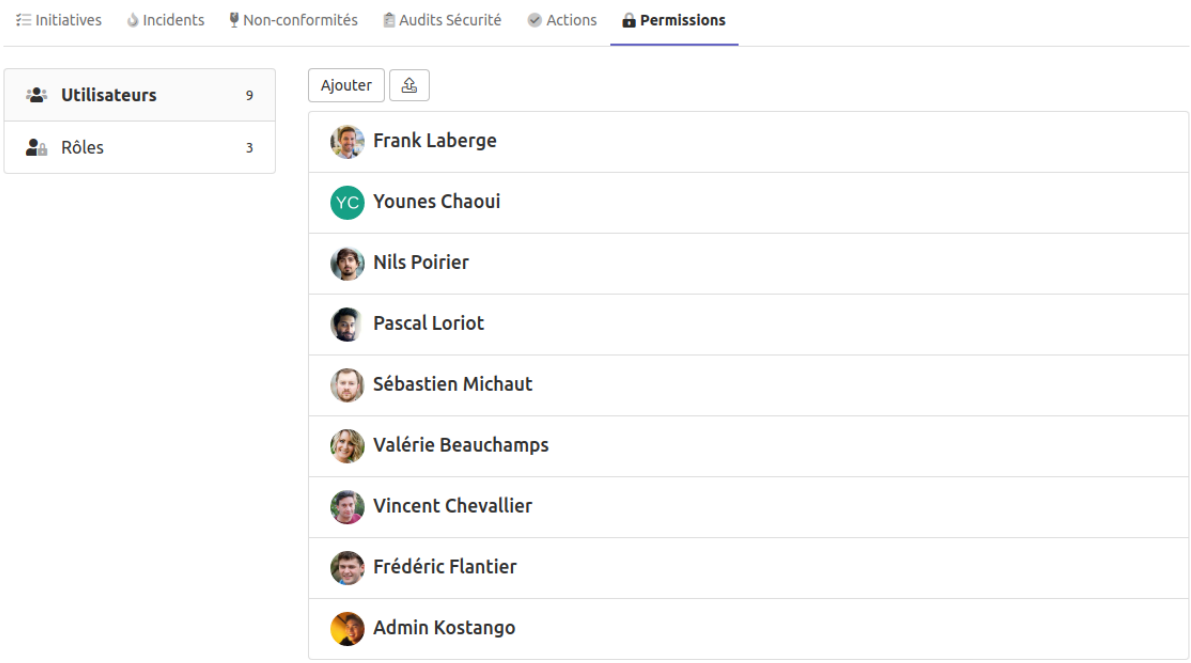


Figure IV.17 – Administration permissions

3.7 Notifications

The notification page shows the latest notifications. We can either mark all the notifications as read or select a notification which will redirect us to its context. This is shown in figure IV.18. These notifications are customizable by setting the notification channel for each event as illustrated in figure IV.19.

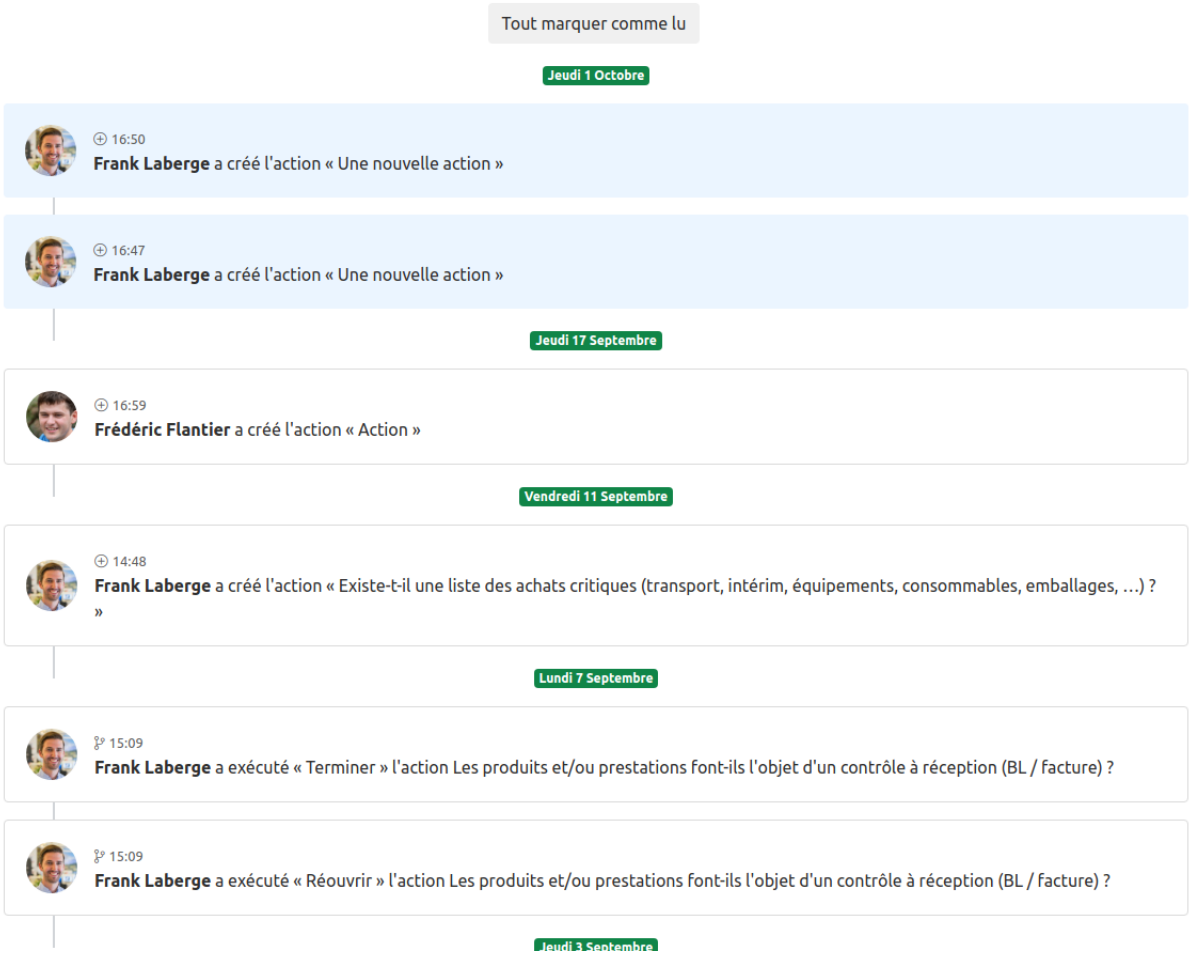


Figure IV.18 – Notifications

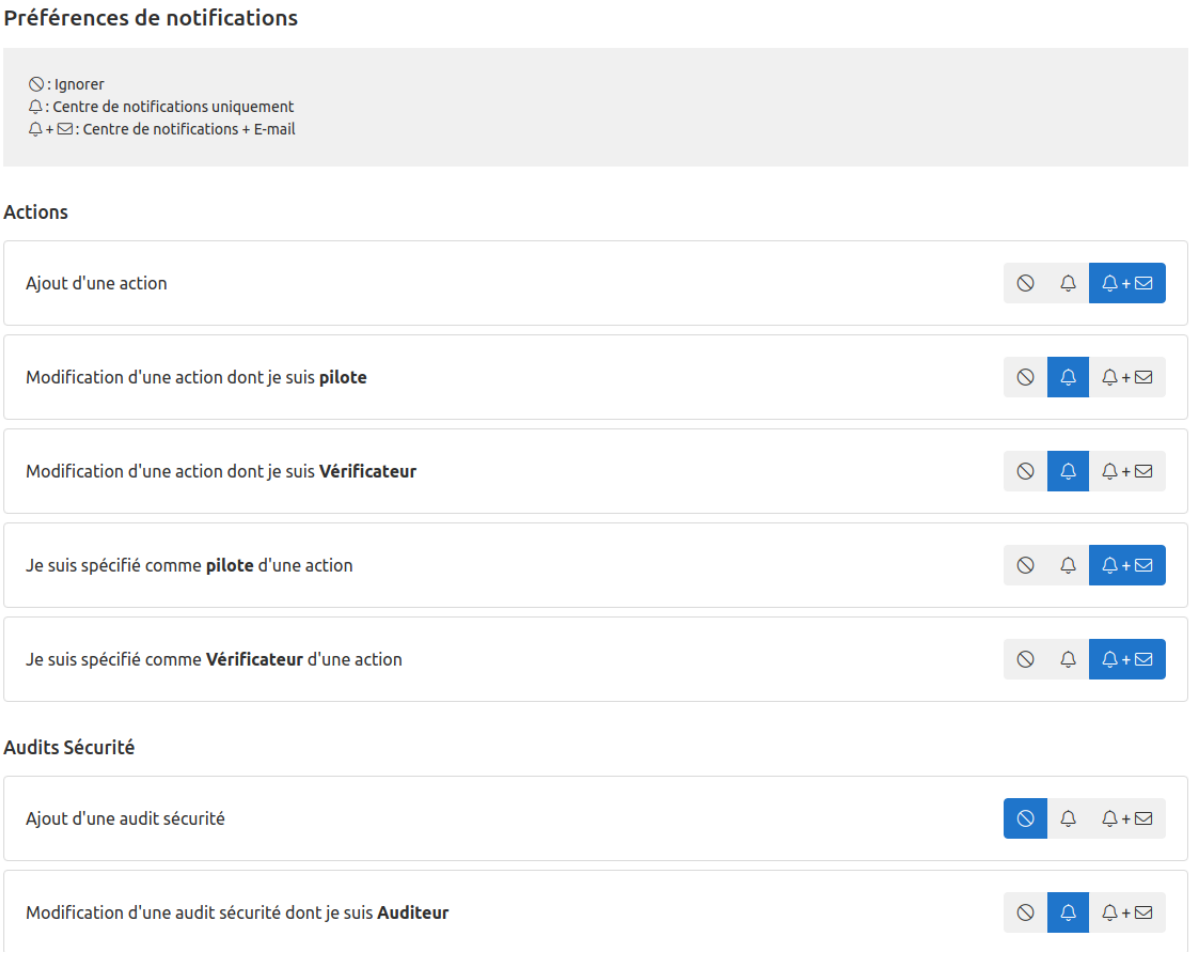


Figure IV.19 – Notification preferences

Conclusion

This final chapter was devoted to the implementation of the solution, for which we detailed the tools we have used and their respective technical architecture. We have also shown screenshots of our solution with detailed illustrations of their execution scenarios. We end this report with a general conclusion and a small glimpse of our future work.

Conclusion and Future Work

This graduation project, completed at Kostango, aimed to rationalize and unite a scattered market by providing an all in one flexible solution that can cover the majority of business cases for many companies that are struggling to keep up with the digitalization wave.

In this report, we have detailed our work methodology carried out to implement our solution.We first presented the project's context and presented the general norms and standards that we aimed to follow. This later layed the foundation for the project's functional scope. We then carefully designed our solution to align with all the requirements. Finally, we've dived into the implementation where we explained our technical choices and showed the final product.

The major challenges encountered in this project are mainly technical. It was essential for us to design and implement parts of the application that were normally provided by default or handled by the database. Everything had to be generic enough to cover an endless amount of business cases whilst keeping configuration needs to a minimum. In a way, our application is a very high level framework that allows us to build a good range of business application. Thus, our main contribution was finding solutions to these technical challenges and setting ourselves apart from the rest of the market. We are glad to contribute to the digitalization wave by substituting the millions of spreadsheets that many businesses still use. We've also did not introduce any barriers in terms of time or cost. Most applications can be fully configured by the Kostango team in 3 to 5 business days.

As for our future work, we aim to add a few more business relevant extensions to our application namely a poll system and timers. We would also like to keep improving that balance between flexibility and business relevance. We plan to do this by simplifying the configuration in the following manner: decreasing flexibility where settings are static and automating the process for repetitive settings. Besides simplifying the configuration, we also want add more flexibility to our entity-relationship model representation system and our automatic workflow system to increase the complexity of the scenarios that can be covered. In terms of DevOps, we plan to have a dedicated server for SCID execution. Finally we're aiming to find the safest and most optimal way to integrate SCID changes in the production environment to keep the integrity of the data.

References

[1] Katie COSTELLO. Ride the 4 Waves of Digital Disruption. https://www.gartner.com/smarterwithgartner/ride-the-4-waves-of-digital-disruption/, (2019). [Online; consulted on Juin 7th 2020]. 1

[2] Olivia LABARRE. Enterprise Resource Planning (ERP). https://www.investopedia.com/terms/e/erp.asp, (2020). [Online; consulted on August 8th 2020]. 1

[3] Kostango. Notre Philosophie. https://kostango.com/notre-philosophie/, (2020). [Online; consulted on Juin 19th 2020]. 4

[4] Benoît HERR. Scrum (Baromètre de satisfaction des utilisateurs de logicielsde gestion). http://www.erp-infos.com/info_article/m/3203/%C3%%2089tude-erp-du-cxp-2017.html, (2017). [Online; consulted on July 18th 2020]. 8

[5] Wikipedia. Scrum (software development). https://en.wikipedia.org/wiki/Scrum_(software_development). [Online; consulted on Juin 26th 2020]. 10

[6] Wikipedia. Source Code in Database. https://en.wikipedia.org/wiki/Source_Code_in_Database. [Online; consulted on August 30th 2020]. 31

[7] Brian LYOHA. What really is the difference between session and token based authentication. https://dev.to/thecodearcher/what-really-is-the-difference-between-session-and-token-based-authentic (2019). [Online; consulted on September 4th 2020]. 33

[8] Django. Django's documentation. https://docs.djangoproject.com/en/3.1/. [Online; consulted on September 23th 2020]. 41, 45

[9] Angular. Angular's documentation. https://angular.io/docs. [Online; consulted on September 25th 2020]. 42

[10] Python. Python's documentation. https://docs.python.org/3/. [Online; consulted on September 24th 2020]. 44

[11] Django Rest Framework. Django Rest Framework documentation. https://www.django-rest-framework.org/. [Online; consulted on September 23th 2020]. 45

[12] PostgreSQL. PostgreSQL's documentation. https://www.postgresql.org/docs/12/index.html. [Online; consulted on September 24th 2020]. 45

[13] NGXS. NGXS documentation. https://www.ngxs.io/. [Online; consulted on September 25th 2020]. 46